

Formation Complète .NET

Les Bases : C# et SQL

Socle Théorique et Fondations du Langage

Objectifs du Module

- Comprendre l'architecture de la plateforme .NET et le fonctionnement du runtime (CLR).
- Installer et configurer un environnement de développement professionnel (Visual Studio / VS Code).
- Maîtriser la syntaxe fondamentale du langage C# et ses structures de contrôle.
- Manipuler avec aisance les collections et structures de données (tableaux, listes, dictionnaires).
- Appréhender les bases de la Programmation Orientée Objet (classes, interfaces, héritage).
- Concevoir et modéliser des bases de données relationnelles avec SQL Server.
- Acquérir une autonomie complète avant d'aborder les projets complexes d'ingénierie logicielle.

Support Pédagogique

Édition Académique & Technique — Version .NET 8+

Kevin Özkaraca

Ebéniste spécialisé : C# - SQL - ASP .NET Core - Angular - WPF - MAUI

2 août 2026

1 Environnement et Premier Programme

1.1 Installation de l'Environnement de Développement

Pour développer efficacement en C#, le choix et la configuration de l'Environnement de Développement Intégré (IDE) sont cruciaux. Deux options principales s'offrent au développeur :

1.1.1 Visual Studio (Environnement Complet)

Visual Studio est l'IDE de référence pour le développement .NET sous Windows.

1. Télécharger la version **Visual Studio Community** (gratuite pour l'apprentissage et l'usage individuel).
2. Lors de l'installation via le *Visual Studio Installer*, cocher impérativement la charge de travail (*workload*) :

Développement .NET Desktop (comprend le SDK .NET, les outils de débogage et C#).

1.1.2 Visual Studio Code (Éditeur Léger & Multiplateforme)

VS Code est un éditeur de code extensible, idéal pour les environnements Linux, macOS ou Windows légers.

1. Télécharger et installer le **SDK .NET 8+** depuis le site officiel Microsoft.
2. Installer **Visual Studio Code**.
3. Ouvrir l'onglet Extensions (**Ctrl+Shift+X**) et installer la suite d'extensions officielles :
 - **C# Dev Kit** (fournit IntelliSense, la gestion de solutions et le débogage avancé).
 - **C#** (support du langage via OmniSharp/Roslyn).

1.2 Qu'est-ce que C#, .NET, CLI et CLR ?

Pour programmer en C# avec rigueur, il est essentiel d'assimiler l'architecture de la plateforme.

1.2.1 Vue d'Ensemble de l'Écosystème

Tableau : Composants clés de l'écosystème Microsoft .NET

Terme	Nature	Description
C#	Langage	Langage de programmation moderne, orienté objet, à typage fort.
.NET	Plateforme	Écosystème réunissant le runtime, les bibliothèques standard et les outils.
Roslyn	Compilateur	Compilateur officiel traduisant le C# en langage intermédiaire (CIL).
CLI	Norme	<i>Common Language Infrastructure</i> — Norme (ECMA-335) régissant .NET.
CLR	Runtime	<i>Common Language Runtime</i> — Moteur d'exécution virtuel exécutant le code.

1.2.2 Définitions des Concepts Clés

Afin de bien appréhender l'écosystème .NET, il est crucial de comprendre la terminologie employée :

- **Langage typé vs non typé (Typage fort)** : Un **langage typé** (comme C# ou Java) exige que chaque donnée en mémoire possède une nature stricte (entier, texte, booléen) définie à l'avance. À l'inverse, un langage à **typage dynamique** (souvent appelé vulgairement "non typé", comme JavaScript ou Python) permet de changer la nature d'une variable à la volée. C# est un langage **fortement typé** : le compilateur vérifie la cohérence des opérations et interdit les actions illogiques (ex : diviser un texte par un entier) avant même l'exécution, garantissant ainsi une grande fiabilité et robustesse des applications.
- **Runtime (Moteur d'exécution)** : Un *runtime* est un environnement logiciel qui s'exécute en arrière-plan et héberge votre programme. Sans le runtime (le CLR pour .NET), votre code compilé serait incompréhensible pour le système d'exploitation. Le runtime est responsable de la traduction finale des instructions pour le processeur physique, de l'allocation mémoire et de la sécurité logicielle.
- **Bibliothèque (Library)** : Une bibliothèque est une collection de code pré-écrit, optimisé et testé, prêt à être intégré dans vos propres projets pour vous éviter de "réinventer la roue". La plateforme .NET inclut la *Base Class Library* (BCL), offrant des milliers de fonctionnalités prêtes à l'emploi (manipulation de fichiers, connexions réseau, algorithmes cryptographiques).
- **Outils (Tools)** : Ce sont les utilitaires qui accompagnent le développeur tout au long du cycle de production logicielle. Quelques exemples incontournables dans .NET :
 - **NuGet** : Le gestionnaire de paquets officiel, permettant de rechercher, télécharger et intégrer des bibliothèques externes tierces.
 - **CLI dotnet (Command Line Interface)** : L'outil permettant de créer, compiler, tester et déployer des applications .NET depuis un terminal.
 - **MSBuild** : Le moteur de construction sous-jacent qui orchestre la compilation des solutions complexes.
- **Norme (Standard / Spécification)** : Une norme est un document théorique et officiel (tel un cahier des charges). Elle ne contient aucun code exécutable, mais dicte les règles absolues de fonctionnement d'un système. Par exemple, la **CLI** est la norme (le plan de construction architectural), tandis que le **CLR** est l'implémentation de cette norme (le moteur réel construit en respectant rigoureusement ce plan).

1.2.3 La CLI (Common Language Infrastructure) : La Norme Théorique

La **CLI** (*Common Language Infrastructure*) est une spécification internationale qui décrit l'architecture nécessaire pour permettre à plusieurs langages haute performance (C#, F#, VB.NET) de s'exécuter sur la même plateforme sans recompilation spécifique à la machine hôte.

La CLI définit deux piliers fondamentaux :

1. **CIL / IL (Common Intermediate Language)** : Le jeu d'instructions universel auquel tous les langages .NET se réduisent après la première phase de compilation.
2. **CTS (Common Type System)** : La spécification unifiée des types de données (garantissant qu'un entier `int` possède la même représentation mémoire en C# et en F#).

1.2.4 Le CLR (Common Language Runtime) : Le Moteur d'Exécution

Le **CLR** est l'implémentation concrète de la spécification CLI par Microsoft. Il agit comme une machine virtuelle hautement optimisée qui prend en charge :

- **La Compilation JIT (Just-In-Time)** : Traduction à la volée du code IL en instructions machines natives (0 et 1) adaptées au processeur cible (x64, ARM64).
- **Le Garbage Collector (GC)** : Gestion automatique de la mémoire et libération des objets inutilisés.
- **La Sécurité de Typage** : Vérification que le code n'accède pas à des zones mémoire non autorisées.

1.2.5 Les 4 Piliers de l'Architecture .NET

Voici le rôle exact de chaque composant dans le cycle de vie de votre code :

- **La CLI (L'Infrastructure)** : C'est le cahier des charges. Elle dicte les règles que tout langage .NET doit respecter (les types de données, la structure du code). Elle garantit que le code C# pourra cohabiter avec du code F#.
- **Roslyn (Le Compilateur)** : C'est l'outil qui lit votre code source C# (.cs). Il vérifie qu'il n'y a pas d'erreurs de syntaxe, puis le traduit en **Code IL** (un code intermédiaire compréhensible par la machine virtuelle).
- **Le CLR (Le Moteur d'Exécution)** : C'est le cœur de .NET. Il prend le Code IL généré par Roslyn et le fait tourner. Il s'occupe de gérer la mémoire (Garbage Collector) et d'assurer la sécurité du programme pendant qu'il tourne.
- **Le JIT (Le Traducteur Final)** : C'est un composant interne au CLR. Juste avant l'exécution, il traduit le Code IL en instructions binaires ultra-rapides, spécifiquement optimisées pour le processeur de la machine sur laquelle le programme tourne (Intel, AMD, ARM).

1.3 Structure d'un Programme C#

Tout programme C# s'articule autour de trois éléments fondamentaux : l'espace de noms (namespace), la classe (class), et le point d'entrée (Main).

1.3.1 Structure Classique (Explicite)

Voici le squelette standard d'une application C# console :

```

1 using System;
2
3 namespace LesBases.Chapitre1; // C# 10 : File-scoped namespace (pas d'
   accolades)
4
5 internal class Program
6 {
7     static void Main(string[] args)
8     {
9         // Point d'entree principal de l'application
10        Console.WriteLine("Bienvenue dans la formation C# !");
11    }
12 }
```

Listing 1 – Structure classique d'un fichier C# (Program.cs)

Explication détaillée du code :

- `using System;` : En C#, le code est organisé en bibliothèques (espaces de noms). Cette ligne dit au compilateur : *"Je veux utiliser les outils fournis par Microsoft, notamment ceux pour écrire dans la console"*. Sans cette ligne, il faudrait écrire `System.Console.WriteLine(...)`.

- `namespace LesBases.Chapitre1` : Un **namespace** (espace de noms) est un dossier virtuel. Il permet de ranger vos classes pour que votre code ne se mélange pas avec celui d'autres développeurs. Si deux classes s'appellent **Program**, le namespace permet de les différencier.
- `internal class Program` : En C# classique, tout doit obligatoirement être contenu dans une "classe". La classe **Program** est traditionnellement le point de départ. Le mot-clé **internal** signifie que cette classe n'est visible que dans ce projet.
- `static void Main(string[] args)` : C'est le point d'entrée critique de l'application. Quand vous lancez le programme, le moteur de .NET cherche très exactement cette méthode **Main** et exécute le code qui se trouve à l'intérieur de ses accolades.

1.3.2 Structure Moderne : Top-Level Statements (C# 9+ / .NET 8)

Dans les versions modernes de C#, le boilerplate (code répétitif) peut être omis pour les scripts simples et les points d'entrée :

```
1 // Les instructions sont directement au niveau racine
2 Console.WriteLine("Bienvenue dans la formation C# moderne !");
```

Listing 2 – Structure moderne avec Top-Level Statements

Quelle structure choisir et comment l'afficher ?

Aujourd'hui, **Microsoft recommande d'utiliser les Top-Level Statements** (la structure moderne) pour les nouveaux projets. C'est plus clair et cela enlève le code inutile pour démarrer. C'est le comportement par défaut quand on crée un nouveau projet .NET.

Cependant, Roslyn cache simplement l'ancienne structure : il génère toujours une classe **Program** et une méthode **Main** en arrière-plan pendant la compilation.

Comment revenir à la structure classique ? Si votre projet nécessite une configuration complexe au démarrage ou si vous préférez l'ancienne méthode, vous pouvez forcer la structure classique :

- Dans Visual Studio, lors de la création du projet, cochez : *"Do not use top-level statements"*.
- En ligne de commande (CLI), tapez : `dotnet new console --use-program-main`.

1.4 Le Cycle de Compilation et d'Exécution

Le processus complet de transformation du code source C# en exécution système suit un pipeline strict en deux phases :

1. Phase 1 : Compilation Statique (Roslyn)

- Analyse syntaxique et sémantique du fichier **.cs**.
- Génération de l'assemblage (fichier **.dll** ou **.exe**).
- L'assemblage contient le **Code IL** et les **Métadonnées**.

2. Phase 2 : Exécution Dynamique (CLR / JIT)

- Chargement de l'assemblage par le CLR.
- Le compilateur **JIT** traduit les blocs IL nécessaires en instructions machines natives.
- Le processeur exécute le code natif.

1.5 Synthèse du Chapitre

Tableau : Récapitulatif des notions clés du Chapitre 1

Concept	A retenir
Roslyn	Compilateur C# qui transforme le code .cs en IL .
Code IL	Langage intermédiaire universel de la plateforme .NET.
CLR	Moteur d'exécution qui gère la mémoire (GC) et compile le code IL via le JIT .
Main()	Point d'entrée de toute application console, explicite ou généré automatiquement par le compilateur (Top-Level Statements).

2 Variables et Types de Données

Les variables constituent le fondement de la programmation impérative. Elles permettent d'allouer de l'espace mémoire pour stocker, lire et manipuler des données tout au long de l'exécution d'un programme. En C#, langage à typage statique fort, chaque variable doit être déclarée avec un type précis, garantissant la sûreté du code dès la compilation.

2.1 Déclaration et Initialisation

La déclaration d'une variable exige de spécifier son **type** suivi de son **identifiant** (nom). L'initialisation consiste à lui attribuer une valeur en mémoire via l'opérateur d'affectation `=`.

```

1 // 1. Déclaration (allocation mémoire statique)
2 int age;
3 string identifiantSysteme;
4
5 // 2. Initialisation (affectation de valeur)
6 age = 30;
7 identifiantSysteme = "SYS_4099";
8
9 // 3. Déclaration et initialisation combinées (Recommandé)
10 double latenceReseau = 19.99;
```

Listing 3 – Déclaration et initialisation standard

2.2 Les Types de Données Primitifs

Le système unifié des types (CTS - Common Type System) de .NET propose de multiples types adaptés aux différents besoins de précision, d'espace mémoire et de logique métier.

2.2.1 Types Fondamentaux

Tableau : Les types de données les plus courants

Type	Nature	Exemple d'utilisation
int	Entier signé 32-bit	int compteurRequetes = 42;
string	Chaîne de caractères (UTF-16)	string nomClient = "Alice";
bool	Booléen (logique)	bool accesAutorise = true;
double	Décimal (64-bit IEEE 754)	double ratio = 3.14159;

2.2.2 Types Spécifiques et Précision

Pour des besoins d'ingénierie particuliers, C# propose des types avec des capacités mémoire étendues ou des gestions d'arrondis spécifiques. Le suffixe de type (**m**, **f**, **L**) est obligatoire lors de l'affectation littérale.

Tableau : Types avancés et suffixes

Type	Spécificité	Exemple
decimal	Haute précision	decimal prix = 19.99m;
float	Simple précision (32-bit)	float delta = 9.81f;
long	Grand entier (64-bit)	long idBaseDonnees = 9876543210L;
char	Caractère unique	char separateur = ',';

Critères de choix : float, double et decimal

- **float** : Utilisé principalement pour la 3D, le rendu graphique ou le traitement du signal (la vitesse de calcul CPU/GPU prime sur l'exactitude extrême).
- **double** : Type décimal par défaut en C#, idéal pour les calculs mathématiques et la trigonométrie.
- **decimal** : Implémente une arithmétique en base 10. Strictement indispensable pour les calculs monétaires et financiers afin d'éviter les pertes de précision liées au format flottant binaire.

2.3 Inférence de Type avec var

L'instruction **var** demande au compilateur de déduire automatiquement le type de la variable en analysant l'expression à droite de l'opérateur d'affectation. Cela allège l'écriture sans sacrifier la sécurité.

```
1 // Le compilateur (Roslyn) déduit 'int'
2 var compteur = 0;
3
4 // Le compilateur déduit 'string'
5 var identifiantSession = "USER_902";
```

Listing 4 – Typage statique implicite avec var

Avertissement Architectural : var n'est pas dynamique

Le mot-clé **var** maintient un typage **statique fort**. Lors de la compilation, l'instruction **var x = 10;** est transformée en **int x = 10;**. Il sera donc impossible d'affecter une chaîne de caractères à **x** ultérieurement.

2.4 Constantes (const) et Champs Immuables (readonly)

Le C# propose deux mécanismes distincts pour déclarer des valeurs non modifiables, chacun avec un comportement fondamentalement différent.

```
1 // 'const' : Valeur résolue et scellée à la COMPILATION
2 // Doit être initialisée immédiatement avec un littéral (pas de 'new',
3   // pas d'appel de méthode)
4 const double Pi = 3.14159;
5
6 // 'readonly' : Valeur scellée à l'EXÉCUTION (après le constructeur)
7 // Peut être initialisée avec le résultat d'un calcul ou d'un appel de m
8   //éthode
9 readonly DateTime _dateCreation = DateTime.UtcNow;
10 readonly string _identifiantMachine;
11
12 public ServiceMonitoring(string machineId)
13 {
14     _identifiantMachine = machineId; // Autorisé uniquement dans le
15     //constructeur
16 }
```

Listing 5 – Déclaration de constantes et champs readonly

Règle : `const` vs `readonly`

- **`const`** : Pour les valeurs universelles connues avant l'exécution (Pi, nombre maximal, codes d'erreur HTTP). Implicitement **`static`**.
- **`readonly`** : Pour les valeurs calculées au démarrage ou injectées par le constructeur (identifiant de session, horodatage de création, configuration chargée depuis un fichier). Non statique par défaut.

2.5 Manipulation des Chaînes de Caractères

En C#, les chaînes (**`string`**) peuvent être combinées (concaténation) à l'aide de l'opérateur `+`. Notez que l'API de base permet également de lire les entrées de l'utilisateur.

```
1 string prenom = "Alan";
2 string nom = "Turing";
3
4 // Concaténation standard
5 string nomComplet = prenom + " " + nom;
6 Console.WriteLine("Utilisateur : " + nomComplet);
7
8 // Récupération d'une saisie dans le flux Standard Input (stdin)
9 Console.WriteLine("Entrez votre code d'accès :");
10 string? codeSaisi = Console.ReadLine(); // 'string?' autorise
    explicitement la valeur null
```

Listing 6 – Concaténation et récupération d'entrées

Note sur `string?` et les *Nullable Reference Types*

Depuis C# 8 (et par défaut depuis .NET 6), les types références (comme **`string`**) ne peuvent plus valoir `null` par défaut, afin d'éradiquer la fameuse `NullReferenceException` en production.

- **`string code = Console.ReadLine();`** générera un avertissement du compilateur, car la console peut renvoyer du vide.
- **`string? codeSaisi`** avec le **point d'interrogation** indique explicitement : *"J'accepte que cette variable puisse être nulle"*. Le compilateur vous obligera ensuite à la vérifier avant de l'utiliser. (Ce mécanisme très puissant sera approfondi au chapitre 18).

2.6 La Portée des Variables (Scope)

La portée d'une variable définit son accessibilité et sa durée de vie en mémoire. C# définit des périmètres d'accès très stricts, indispensables à l'encapsulation.

Tableau : Les niveaux de portée en C#

Type de portée	Zone de déclaration	Durée de vie
Locale	Dans une méthode ou un bloc <code>{ }</code>	Détruite à la fin du bloc d'exécution.
Champ d'instance	Dans la classe (hors méthode)	Détruite avec l'instance de l'objet (via le GC).
Statique	Dans la classe (avec <code>static</code>)	Réside en mémoire jusqu'à la fermeture du processus.

2.6.1 Démonstration des Portées et du Masquage (Shadowing)

```
1 using System;
2
3 class ServeurWeb
4 {
5     // 1. Champ statique : Partagé par toutes les instances de
    ServeurWeb en mémoire
6     private static int _requetesGlobales = 0;
7
8     // 2. Champ d'instance : Propre à une instance spécifique du serveur
9     private string _adresseIp;
10
11     public ServeurWeb(string adresseIp)
12     {
13         _adresseIp = adresseIp;
14     }
15
16     public void TraiterRequete(string _adresseIp)
17     {
18         // 3. Masquage (Shadowing) : Le paramètre masque le champ d'
    instance
19         // (Note pédagogique : l'usage d'un '_' pour un paramètre est un
    anti-pattern,
20         // utilisé ici EXCLUSIVEMENT pour démontrer le masquage).
21
22         // Incrémentation de la variable statique partagée
23         _requetesGlobales++;
24
25         // 4. Variable locale : Isolée dans le bloc de cette méthode
26         int taillePayload = 1024;
27
28         // Pour accéder au champ d'instance masqué, l'usage de 'this'
    est obligatoire
29         Console.WriteLine($"Connexion de {_adresseIp} sur l'hôte {this.
    _adresseIp}");
30     }
31 }
```

Listing 7 – Gestion de la portée, variables statiques et masquage

2.7 Le Type dynamic

Contrairement à `var`, le mot-clé `dynamic` désactive intégralement la vérification statique du compilateur. Le type n'est résolu qu'au moment de l'exécution (Runtime).

```
1 dynamic variableDynamique = 10;
2 variableDynamique = "Transformation en chaîne de caractères"; // Aucune
    erreur de compilation
```

Listing 8 – Désactivation du typage avec `dynamic`

L'usage de `dynamic` doit être rigoureusement limité à des cas d'ingénierie spécifiques (interopérabilité COM, réflexion, traitement de structures JSON non typées) car il annule toute la sécurité offerte par le langage.

2.8 Conversion de Types (Casting)

La conversion de types est une opération courante, notamment lors de l'intégration de flux de données externes.

```

1 // 1. Conversion Implicite (Sûre - Garantie sans perte d'information)
2 int entier = 10;
3 double reel = entier; // L'entier (32-bit) s'intègre sans perte dans le
   decimal (64-bit)
4
5 // 2. Conversion Explicite ou Cast (Avertissement : risque de troncature
   )
6 double valeurMesuree = 9.7;
7 int valeurTronquee = (int)valeurMesuree; // Vaut 9 (la fraction décimale
   est détruite)
8
9 // 3. Conversion par la classe utilitaire Convert (arrondit au lieu de
   tronquer)
10 int valeurArrondie = Convert.ToInt32(valeurMesuree); // Vaut 10 (arrondi
   bancaire)
11
12 // 4. Conversion par Analyse (Parsing : Chaîne vers Numérique)
13 string donneesReseau = "1024";
14 int portServeur = int.Parse(donneesReseau);

```

Listing 9 – Mécanismes de conversion en C#

2.9 Bonnes Pratiques de Codage

Règles strictes de développement C#

- **Variables non initialisées** : Le compilateur C# refuse catégoriquement l'utilisation d'une variable locale non assignée (**Use of unassigned local variable**). Les champs de classe, eux, prennent par défaut la valeur 0 ou null.
- **Confusion Affectation / Comparaison** : Ne jamais confondre l'opérateur d'affectation = avec l'opérateur d'égalité logique ==.
- **Déclarations en ligne** : L'écriture `int a = 1, b = 2;` est permise par le compilateur mais prohibée par les conventions de Clean Code. L'architecture logicielle exige une déclaration par ligne pour préserver la lisibilité de l'historique de versionnage (Git).

3 Conventions de Nommage, Formatage et Clean Code

Écrire du code informatique ne se limite pas à produire une séquence d'instructions exécutables par le compilateur. Dans l'ingénierie logicielle professionnelle, le code C# doit être **lisible**, **maintenable** et **homogène**. Les conventions de style constituent le langage commun facilitant la collaboration au sein des équipes de développement.

P1 02 b.tex

3.1 Conventions de Nommage Officielles

Le nommage d'un élément logiciel doit rendre le code auto-documenté. Microsoft définit des règles précises de casse selon la nature et la portée de chaque symbole.

3.1.1 Les Styles de Casse en C#

Tableau : Règles de casse officielles C# (Microsoft Design Guidelines)

Style	Format	Exemple	Cas d'utilisation
PascalCase	Majuscule à chaque mot	CalculerPrixTotal	Classes, structures, enums, méthodes, propriétés, événements.
camelCase	1 ^{er} mot en minuscule	prixUnitaire	Variables locales, paramètres de méthodes.
_camelCase	camelCase précédé de _	_compteurClient	Champs privés d'une classe (<i>private fields</i>).
IPascalCase	PascalCase avec préfixe I	IDisposable	Interfaces.
PascalCase	Majuscule à chaque mot	NombreMaxEssais	Constantes (<code>const</code> et <code>readonly</code>).

3.1.2 Exemples de Conformité Syntaxique

```
1 // Interface : Prefix 'I' + PascalCase
2 public interface ICalculable
3 {
4     // Propriete : PascalCase
5     decimal Total { get; }
6 }
7
8 // Classe : PascalCase
9 public class GestionnaireCommande : ICalculable
10 {
11     // Constante : PascalCase
12     public const int LimiteArticles = 100;
13
14     // Champ prive : _camelCase
15     private readonly string _identifiantCommande;
16     private int _nombreArticles;
17
18     // Propriete publique : PascalCase
19     public decimal Total { get; private set; }
20
21     // Constructeur : parametre en camelCase
22     public GestionnaireCommande(string identifiantCommande)
23     {
24         _identifiantCommande = identifiantCommande;
25     }
26
27     // Methode : PascalCase, parametre : camelCase, variable locale :
28     // camelCase
29     public void AjouterArticle(decimal prixUnitaire, int quantite)
30     {
31         decimal sousTotal = prixUnitaire * quantite;
32         Total += sousTotal;
33         _nombreArticles += quantite;
34     }
35 }
```

Listing 10 – Conventions de nommage recommandées en C#

Pièges de nommage à éviter

- **Ne pas utiliser le snake_case** (`prix_total`) : réservé à d'autres langages (Python, Rust).
- **Éviter la notation hongroise** (`strNom`, `intAge`) : le typage C# est déjà statique et explicite.
- **Omettre le préfixe I sur les interfaces** : rend la distinction impossible avec les classes abstraites.

3.2 Conventions de Formatage (Style Allman)

Le formatage régit l'organisation visuelle du code source. C# adopte historiquement le style d'accolades **Allman**.

3.2.1 Règles Fondamentales d'Organigramme Visuel

1. **Accolades Allman** : L'accolade ouvrante `{` se place **toujours sur sa propre ligne**, alignée verticalement avec la structure parente.
2. **Indentation** : Strictement 4 espaces par niveau de bloc (ne pas mélanger tabulations et espaces).
3. **Espacement des opérateurs** : Un espace unique de part et d'autre des opérateurs binaires (`=`, `+`, `==`, `&&`).

```
1 // [CORRECT] Style Allman officiel C#
2 public void InscrireUtilisateur(string nom)
3 {
4     if (string.IsNullOrEmpty(nom))
5     {
6         throw new ArgumentException("Nom invalide", nameof(nom));
7     }
8 }
9
10 // [INCORRECT] Style K&R (Anti-pattern en C#)
11 public void InscrireUtilisateur(string nom) {
12     if (string.IsNullOrEmpty(nom)) {
13         throw new ArgumentException("Nom invalide", nameof(nom));
14     }
15 }
```

Listing 11 – Comparaison de formatage : Allman (C#) vs K&R (Java/JS)

3.3 Organisation Interne d'une Classe C#

Une classe lisible regroupe ses membres selon un ordre standardisé reconnu par la communauté.

Tableau : Ordre d'agencement recommandé des membres d'une classe

Ordre	Catégorie de Membre	Description
1	Constantes	public const et private const.
2	Champs privés	Champs d'instance d'encapsulation (<code>_camelCase</code>).
3	Constructeurs	Initialisateurs d'instance.
4	Propriétés publiques	Accesseurs de données (<code>get</code> , <code>set</code> , <code>init</code>).
5	Méthodes publiques	Points d'entrée de la logique métier.
6	Méthodes privées	Méthodes utilitaires internes d'assistance.

```

1 namespace LesBases.Services;
2
3 public class ServiceCompteBancaire
4 {
5     // 1. Constantes
6     private const decimal SoldeMinimumAutorise = -500.00m;
7
8     // 2. Champs privés
9     private readonly string _numeroCompte;
10    private decimal _solde;
11
12    // 3. Constructeurs
13    public ServiceCompteBancaire(string numeroCompte, decimal
soldeInitial)
14    {
15        _numeroCompte = numeroCompte;
16        _solde = soldeInitial;
17    }
18
19    // 4. Proprietes publiques
20    public string NumeroCompte => _numeroCompte;
21    public decimal Solde => _solde;
22
23    // 5. Methodes publiques
24    public void Depositer(decimal montant)
25    {
26        ValiderMontantPositif(montant);
27        _solde += montant;
28    }
29
30    // 6. Methodes privees d'assistance
31    private static void ValiderMontantPositif(decimal montant)
32    {
33        if (montant <= 0)
34        {
35            throw new ArgumentOutOfRangeException(nameof(montant), "Le
montant doit etre positif.");
36        }
37    }
38 }

```

Listing 12 – Exemple de structure canonique d'un service C#

3.4 Inférence de Type (var) vs Type Explicite

Le mot-clé `var` (introduit en C# 3) permet l'inférence de type à la compilation. Son utilisation est encadrée par des critères d'évidence visuelle.

Règle d'or de l'inférence var

Utiliser `var` uniquement lorsque le type retourné est **immédiatement explicite** à la lecture du côté droit de l'affectation (opérateur `new` ou littéral). Dans le cas contraire, conserver le type explicite pour maintenir la clarté.

```
1 // [BON] Le type est evident via l'instanciation directe ou le littéral
2 var nomUtilisateur = "Alice"; // string
3 var listeClients = new List<string>(); // List<string>
4 var dictionnaire = new Dictionary<int, string>(); // Evite la
    repetition inutile
5
6 // [MAUVAIS] Le type est masque par l'appel de methode
7 var resultat = CalculerIndiceForme(); // De quel
    type est resultat ? int ? double ? CustomType ?
8
9 // [CORRECT] Type explicite quand l'intention doit être clarifiée
10 decimal indiceForme = CalculerIndiceForme();
```

Listing 13 – Usage correct et incorrect de var

3.5 Contrôle de Flux et Retours Anticipés (*Early Return*)

Pour éviter l'empilement excessif de conditions imbriquées (syndrome de la pyramide d'indentation), C# préconise la technique des **retours anticipés**.

```
1 // [MAUVAIS] Imbrication profonde et illisible
2 public string TraiterCommande(Commande cmd)
3 {
4     if (cmd != null)
5     {
6         if (cmd.EstValidee)
7         {
8             if (cmd.StockDisponible)
9             {
10                 return "Commande expediee";
11             }
12             else
13             {
14                 return "Rupture de stock";
15             }
16         }
17         else
18         {
19             return "Commande non validee";
20         }
21     }
22     return "Erreur commande nulle";
23 }
24
25 // [EXCELLENT] Structure avec retours anticipés (Early Return)
26 public string TraiterCommandePropre(Commande cmd)
27 {
28     if (cmd == null) return "Erreur commande nulle";
29     if (!cmd.EstValidee) return "Commande non validee";
30     if (!cmd.StockDisponible) return "Rupture de stock";
31
32     return "Commande expediee";
33 }
```

33 }

Listing 14 – Réfactorisation d’une méthode complexe par Early Return

3.6 Gestion Rigoureuse des Exceptions

La gestion des erreurs doit respecter la chaîne de rétropropagation des exceptions sans altérer la pile d’appels (*stack trace*).

Règle d’or : `throw;` vs `throw ex;`

Lors du re-lancement d’une exception dans un bloc `catch`, utiliser strictement `throw;` sans argument. L’instruction `throw ex;` réinitialise la pile d’appels au niveau du `catch`, masquant l’origine exacte du plantage.

```
1 try
2 {
3     ExecuterTraitementReseau();
4 }
5 catch (TimeoutException ex)
6 {
7     // [BON] Conserve l'intégralité de la stack trace originale
8     Logger.LogError(ex, "Timeout reseau detecte");
9     throw;
10 }
11 catch (Exception ex)
12 {
13     // [INCORRECT] Ecraser la stack trace originale
14     // throw ex; <-- NE JAMAIS FAIRE CELA
15 }
```

Listing 15 – Bonnes pratiques de re-lancement d’exceptions

3.7 Outillage Automatisé (.editorconfig & Roslyn)

Afin de garantir le respect des conventions sans effort manuel, l’écosystème .NET utilise des fichiers de configuration standardisés à la racine du dépôt.

3.7.1 Fichier de Configuration .editorconfig

```
1 root = true
2
3 [*.*]
4 # Indentation
5 indent_style = space
6 indent_size = 4
7
8 # Regles de nommage des champs privés
9 dotnet_naming_rule.private_fields_should_have_underscore.severity =
    warning
10 dotnet_naming_rule.private_fields_should_have_underscore.symbols =
    private_fields
11 dotnet_naming_rule.private_fields_should_have_underscore.style =
    prefix_underscore
12
13 dotnet_naming_symbols.private_fields.applicable_kinds = field
```



```

14 dotnet_naming_symbols.private_fields.applicable_accessibilities =
    private
15
16 dotnet_naming_style.prefix_underscore.required_prefix = _
17 dotnet_naming_style.prefix_underscore.capitalization = camel_case
18
19 # Respect obligatoire des accolades Allman
20 csharp_prefer_braces = true:warning

```

Listing 16 – Exemple de fichier .editorconfig standard pour projet C#

3.7.2 Commande de Formatage CLI

La CLI .NET intègre un outil natif pour corriger le formatage du projet entier :

```

1 # Reformate automatiquement tous les fichiers de la solution selon le .
  editorconfig
2 dotnet format
3
4 # Verification sans modification (ideal pour les pipelines CI/CD)
5 dotnet format --verify-no-changes

```

Listing 17 – Commandes dotnet format

3.8 Synthèse des Conventions

Tableau : Tableau récapitulatif des règles d'ingénierie C#

Élément	Convention Officielle	Exemple Validé
Classe / Structure	PascalCase	class CompteUtilisateur
Interface	I + PascalCase	interface IRepository
Méthode / Propriété	PascalCase	public void Sauvegarder()
Variable locale	camelCase	int totalElements = 0;
Champ privé	_camelCase	private string _cleApi;
Accolades	Style Allman	Première ligne dédiée
Re-throw exception	throw;	Préserve la pile d'appels

4 Architecture des Types de Variables et Mémoire

En programmation C#, chaque variable possède un **type** statique fondamental qui définit non seulement l'espace mémoire alloué, mais également la nature des opérations permises par le processeur. Comprendre la dichotomie entre les types valeur et les types référence est essentiel pour maîtriser la gestion mémoire du Common Language Runtime (CLR).

4.1 Types Valeur et Types Référence

Le CLR divise la mémoire de l'application en deux zones distinctes : la **Pile d'exécution** (*Stack*) et le **Tas managé** (*Managed Heap*). La classification d'un type détermine la zone d'allocation de la donnée.

4.1.1 Les Types Valeur (*Value Types*)

Les variables locales de type valeur sont généralement stockées directement sur la **pile** (*Stack*), mais un type valeur peut également résider sur le **tas** lorsqu'il est contenu dans un objet (champ d'une classe) ou dans un tableau. Lorsqu'un type valeur est assigné à une nouvelle variable, le système effectue une **copie complète** de la valeur. Les deux variables opèrent alors sur des emplacements mémoire strictement indépendants.

```

1 // Allocation sur la pile (Stack)
2 int ageUtilisateur = 25;
3
4 // Copie bit à bit dans un nouvel emplacement mémoire
5 int ageSauvegarde = ageUtilisateur;
6
7 // La modification de la copie n'a aucun effet sur l'original
8 ageSauvegarde = 30;
9
10 Console.WriteLine(ageUtilisateur); // Affiche 25
11 Console.WriteLine(ageSauvegarde); // Affiche 30

```

Listing 18 – Comportement en mémoire des types valeur (pile)

4.1.2 Les Types Référence (*Reference Types*)

Les types référence ne contiennent pas directement la donnée. Ils stockent une **adresse mémoire** (un pointeur managé) pointant vers l'objet réel alloué dans le **Tas** (*Heap*). Lors de l'affectation d'un type référence à une autre variable, seule l'adresse mémoire est copiée. Les deux variables pointent alors vers la **même instance** sous-jacente.

```

1 // Allocation de l'objet dans le tas, et du pointeur dans la pile
2 int[] notesExamen = new int[] { 10, 15, 20 };
3
4 // Copie de l'adresse mémoire : les deux variables pointent vers le même
  objet
5 int[] copieNotes = notesExamen;
6
7 // La modification via la référence secondaire altère l'objet partagé
8 copieNotes[0] = 5;
9
10 Console.WriteLine(notesExamen[0]); // Affiche 5

```

Listing 19 – Comportement en mémoire des types référence (tas)

Piège de l'ingénierie : L'Exception de Référence Nulle

Tenter d'accéder aux membres d'un type référence qui ne pointe vers aucune instance (état `null`) provoquera l'arrêt brutal du programme via une `NullPointerException`. Il est impératif d'initialiser les instances ou de vérifier la nullité avant accès.

4.2 Familles de Types Primitifs

4.2.1 Les Types Entiers (`int`, `long`, `byte`)

Les entiers stockent des nombres sans composante fractionnaire. Le choix du type dépend de l'espace mémoire souhaité et des limites mathématiques attendues.

- `byte` : Entier non signé de 8 bits (0 à 255). Hautement optimisé pour la manipulation de flux binaires ou le traitement de trames réseau.
- `int` : Entier signé de 32 bits (−2 147 483 648 à 2 147 483 647, soit $\approx \pm 2,14 \times 10^9$). Le type standard par défaut pour les boucles et les compteurs.
- `long` : Entier signé de 64 bits. Indispensable pour l'architecture Big Data, les identifiants de bases de données massives ou le calcul astronomique (suffixe `L` recommandé).

L'Overflow (Dépassement de Capacité)

Assigner une valeur dépassant la limite physique du type (ex : assigner 256 à un `byte`) provoque un *Overflow*. Selon la configuration du compilateur, cela entraînera une `OverflowException` ou un comportement cyclique silencieux entraînant des bugs critiques dans la logique d'état.

4.2.2 Les Types Flottants (`float`, `double`, `decimal`)

Les types à virgule flottante implémentent la norme IEEE 754 pour stocker des valeurs décimales. La précision de l'ingénierie détermine le choix du type.

- `float` (32-bit, suffixe `f`) : Optimisé pour les calculs vectoriels, la physique 3D et le rendu GPU où la vitesse de calcul prime sur l'exactitude extrême au micron près.
- `double` (64-bit) : Le standard C# par défaut pour les calculs scientifiques et géométriques.
- `decimal` (128-bit, suffixe `m`) : Implémentation en base 10. Ce type requiert plus de cycles CPU mais évite toute aberration d'arrondi binaire. Son usage est **strictement obligatoire** pour les transactions financières et monétaires.

4.2.3 Les Types Textuels (`char` et `string`)

- `char` : Type valeur représentant un unique caractère Unicode UTF-16 (encadré par des guillemets simples ' ').
- `string` : Type référence immuable représentant une séquence de caractères (encadrée par des guillemets doubles " "). Toute opération de modification (concaténation) alloue secrètement un nouvel objet en mémoire géré par le Garbage Collector.

L'ingénierie logicielle moderne favorise l'utilisation de l'**Interpolation de chaînes** (`$""`) pour des raisons de lisibilité et de maintenance, évitant ainsi les concaténations verbeuses.

```
1 string adresseServeur = "192.168.1.1";
2 int portReseau = 8080;
3
4 // Injection directe des variables via l'interpolation (préfixe $)
```

```

5 string requeteLog = $"Connexion entrante sur le serveur {adresseServeur
  }:{portReseau}";
6
7 Console.WriteLine(requeteLog);

```

Listing 20 – Interpolation de chaînes de caractères

4.2.4 Les Types Temporels (DateTime, DateOnly, TimeSpan)

La gestion du temps est omniprésente en ingénierie logicielle (horodatage de logs, expiration de tokens, calcul de durées de traitement). Le framework .NET fournit des structures spécialisées.

```

1 // DateTime : Date ET heure (type valeur, struct)
2 DateTime maintenant = DateTime.Now;           // Heure locale du serveur
3 DateTime instantUtc = DateTime.UtcNow;        // Standard international (
   recommandé pour les APIs)
4 DateTime datePersonnalisee = new DateTime(2026, 8, 2, 14, 30, 0);
5
6 // DateOnly (C# 10 / .NET 6+) : Date pure sans composante horaire
7 DateOnly dateNaissance = new DateOnly(1995, 3, 15);
8 DateOnly aujourd'hui = DateOnly.FromDateTime(DateTime.Now);
9
10 // TimeSpan : Représente une durée (un intervalle de temps)
11 TimeSpan dureeTraitement = TimeSpan.FromMilliseconds(1500);
12 TimeSpan delaiMax = new TimeSpan(0, 5, 0); // 5 minutes
13
14 // Arithmétique temporelle
15 DateTime expiration = DateTime.UtcNow.AddHours(24);
16 TimeSpan ecart = expiration - DateTime.UtcNow;
17 Console.WriteLine($"Le token expire dans {ecart.TotalHours:F1} heures.")
   ;
18
19 // Formatage pour affichage ou sérialisation
20 string logTimestamp = instantUtc.ToString("yyyy-MM-dd HH:mm:ss");

```

Listing 21 – Manipulation des types temporels

Piège : DateTime.Now vs DateTime.UtcNow

`DateTime.Now` retourne l'heure locale du serveur (dépend du fuseau horaire). Si votre application est déployée sur des serveurs répartis géographiquement (cloud Azure, AWS), les horodatages seront incohérents. **Règle d'or** : Utilisez systématiquement `DateTime.UtcNow` pour stocker et comparer des instants, et convertissez en heure locale uniquement lors de l'affichage à l'utilisateur final.

4.2.5 Génération de Nombres Aléatoires (Random)

La classe `System.Random` fournit un générateur de nombres pseudo-aléatoires. En .NET 6+, la propriété statique `Random.Shared` offre une instance thread-safe partagée, éliminant le besoin de créer manuellement des instances.

```

1 // .NET 6+ : Utiliser l'instance partagée (thread-safe)
2 int identifiantTemporaire = Random.Shared.Next(1000, 10000); // Entier
   entre 1000 et 9999 inclus
3 double facteurAleatoire = Random.Shared.NextDouble();        // Double
   entre 0.0 et 1.0
4
5 // Cas d'usage : simuler un délai de retry aléatoire (jitter)

```

```

6 int delaiRetryMs = Random.Shared.Next(100, 2000);
7 Console.WriteLine($"Nouvelle tentative dans {delaiRetryMs}ms...");

```

Listing 22 – Génération de nombres aléatoires

4.3 L'État Indéfini : Les Types Nullable

Les types valeur (`int`, `bool`) ne peuvent, par définition structurelle, pas contenir `null`. Or, dans l'interaction avec des bases de données relationnelles ou des API JSON, une donnée peut légitimement être absente. C# permet d'envelopper un type valeur dans une structure autorisant le vide via l'opérateur suffixe `?`.

```

1 // L'ajout de '?' permet à un type valeur d'accepter l'état 'null'
2 int? identifiantSessionFacultatif = null;
3
4 // L'opérateur de coalescence nulle '??' vérifie si la variable de
   gauche est null.
5 // Si oui, il injecte la valeur de repli située à sa droite.
6 int idSessionActif = identifiantSessionFacultatif ?? 9999;
7
8 Console.WriteLine($"Session de secours allouée : {idSessionActif}"); //
   Affiche 9999

```

Listing 23 – Types nullable et opérateur de coalescence

4.4 Conversions Stratégiques : Le Parsing

Il est très fréquent d'ingérer des flux textuels (`string`) nécessitant une transformation en types mathématiques exploitables. L'utilisation de `TryParse` est la norme de l'industrie pour prévenir les crashes liés aux entrées utilisateur malformées.

```

1 string chargeUtileReseau = "CORROMPU_1024";
2
3 // TryParse tente la conversion sans jamais lever d'exception bloquante.
4 // Il retourne un booléen et injecte le résultat dans le paramètre 'out'
   en cas de succès.
5 if (int.TryParse(chargeUtileReseau, out int octetsRecus))
6 {
7     Console.WriteLine($"Traitement des paquets : {octetsRecus} octets.");
8 }
9 else
10 {
11     Console.WriteLine("Erreur de protocole : charge utile non-numérique
   détectée.");
12 }

```

Listing 24 – Conversion sécurisée via `TryParse`

4.5 Synthèse du Chapitre

Tableau : Rétrospective de l'architecture des types

Concept	Règle de l'Ingénierie
Stack (Pile)	Zone mémoire ultra-rapide hébergeant les <i>Types Valeur</i> .
Heap (Tas)	Zone mémoire gérée par le GC hébergeant les instances des <i>Types Référence</i> .
<code>decimal</code>	Le seul type flottant autorisé pour les opérations financières.
<code>TryParse</code>	Approche architecturale requise pour toute conversion d'entrée externe incertaine.

5 Les Opérateurs et Expressions

En C#, les opérateurs sont des symboles syntaxiques spécifiques qui indiquent au compilateur de réaliser des opérations mathématiques, logiques ou d'affectation sur des opérandes (variables ou valeurs). La maîtrise des opérateurs est indispensable pour manipuler les données et contrôler le flux d'exécution.

5.1 Opérateurs Arithmétiques

Les opérateurs arithmétiques effectuent des calculs mathématiques standard. Le comportement de la division dépend du type de ses opérandes : si les deux sont des entiers, le résultat est une division entière (tronquée).

Tableau : Opérateurs Arithmétiques

Opérateur	Opération	Exemple	Résultat
+	Addition	5 + 3	8
-	Soustraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division entière	10 / 3	3
%	Modulo (reste)	10 % 3	1

```

1 int a = 10, b = 3;
2 Console.WriteLine(a / b);           // Affiche : 3
3 // Le transtypage (cast) explicite force une division à virgule
  flottante
4 Console.WriteLine((double)a / b); // Affiche : 3.3333333333333335

```

Listing 25 – Division entière vs Division flottante

5.2 Opérateurs d'Affectation

L'opérateur d'affectation de base est le signe =. C# propose également des opérateurs d'affectation composés, qui combinent une opération arithmétique ou binaire avec l'affectation, permettant d'alléger la syntaxe.

Tableau : Opérateurs d'Affectation Composés

Opérateur	Syntaxe équivalente	Exemple
+=	a = a + b	valeur += 3;
-=	a = a - b	valeur -= 3;
*=	a = a * b	valeur *= 3;
/=	a = a / b	valeur /= 3;
%=	a = a % b	valeur %= 3;

5.3 Opérateurs d'Incrémentement et Décrémentement

Ces opérateurs unaires augmentent ou diminuent la valeur d'une variable numérique de 1. L'emplacement de l'opérateur (préfixe ou suffixe) détermine le moment de l'évaluation par le CLR.

- **Préfixé** (++a) : La variable est incrémentée **avant** l'évaluation de l'expression.
- **Suffixé** (a++) : La variable est évaluée dans l'expression courante, puis incrémentée **après**.

```

1 int index = 5;
2 Console.WriteLine(index++); // Affiche 5, puis index devient 6
3 Console.WriteLine(++index); // index devient 7, puis s'affiche (7)

```

Listing 26 – Incrémentation préfixée et suffixée

5.4 Opérateurs de Comparaison (Relationnels)

Ils évaluent la relation entre deux opérandes et retournent systématiquement une valeur de type `bool` (`true` ou `false`). Ils constituent la base des structures conditionnelles.

Tableau : Opérateurs de Comparaison

Opérateur	Signification	Exemple
<code>==</code>	Égal à	<code>a == b</code>
<code>!=</code>	Différent de	<code>a != b</code>
<code>></code>	Strictement supérieur	<code>a > b</code>
<code><</code>	Strictement inférieur	<code>a < b</code>
<code>>=</code>	Supérieur ou égal	<code>a >= b</code>
<code><=</code>	Inférieur ou égal	<code>a <= b</code>

Erreur Courante : Affectation vs Égalité

Une erreur classique d'ingénierie consiste à utiliser l'opérateur d'affectation `=` (un seul signe égal) au lieu de l'opérateur de comparaison `==` dans une condition. Le compilateur C# bloquera cette erreur (`Cannot implicitly convert type...`) si le résultat n'est pas un booléen.

5.5 Opérateurs Logiques

Les opérateurs logiques permettent de combiner plusieurs expressions booléennes. C# implémente l'évaluation "court-circuit" : si le premier opérande suffit à déterminer le résultat final, le second opérande n'est pas évalué.

Tableau : Opérateurs Logiques

Opérateur	Nom	Comportement Court-Circuit
<code>&&</code>	ET Logique (<i>AND</i>)	Si l'opérande gauche est <code>false</code> , l'opérande droit est ignoré (le résultat sera <code>false</code>).
<code> </code>	OU Logique (<i>OR</i>)	Si l'opérande gauche est <code>true</code> , l'opérande droit est ignoré (le résultat sera <code>true</code>).
<code>!</code>	NON Logique (<i>NOT</i>)	Inverse la valeur booléenne (opérateur unaire).

```

1 bool accesBdd = true;
2 bool compteVerrouille = false;
3
4 // Vérifie si l'accès est disponible ET que le compte n'est PAS
   verrouillé
5 if (accesBdd && !compteVerrouille)
6 {
7     Console.WriteLine("Autorisation accordée.");
8 }

```

Listing 27 – Évaluation logique conditionnelle

5.6 Opérateurs Avancés et Bit à Bit (*Bitwise*)

Les opérateurs de bits manipulent directement la représentation binaire (les bits) d'une donnée en mémoire. Bien qu'essentiels en cryptographie, programmation réseau ou systèmes embarqués, ils sont rarement utilisés dans des applications de gestion standard.

- **ET bit à bit (&)** : 1 si les deux bits sont à 1.
- **OU bit à bit (|)** : 1 si au moins un des bits est à 1.
- **OU exclusif (^)** : 1 si les bits sont différents.
- **Décalage à gauche («)** : Décale les bits à gauche (multiplie par 2).
- **Décalage à droite (»)** : Décale les bits à droite (divise par 2).

5.7 Opérateurs Spécifiques au Typage et Contrôle

C# intègre des opérateurs haut niveau pour gérer les types et l'état `null`.

- **Opérateur ternaire (?:)** : Raccourci syntaxique pour une condition `if...else` sur une seule ligne. Syntaxe : `condition ? valeur_si_vrai : valeur_si_faux`.
- **Opérateur de coalescence (??)** : Retourne l'opérande de gauche s'il n'est pas `null`, sinon retourne l'opérande de droite.
- **is** : Évalue si un objet est compatible avec un type donné, au moment de l'exécution (Runtime).
- **as** : Tente un transtypage (cast) en toute sécurité. Si l'opération échoue, retourne `null` au lieu de lever une exception. Fonctionne avec les types référence et les types valeur nullable (`T?`).

```
1 object donneeInconnue = 42;
2
3 // Vérification de type (Pattern matching)
4 if (donneeInconnue is int nombre)
5 {
6     Console.WriteLine($"C'est un entier de valeur {nombre}");
7 }
8
9 // Transtypage sécurisé (fonctionne uniquement avec les types référence
10 // ou nullable)
11 string? texteConnu = donneeInconnue as string;
12 // texteConnu vaut null car donneeInconnue est un entier
```

Listing 28 – Utilisation de l'opérateur `is` et `as`

5.8 Priorité des Opérateurs

Lorsqu'une expression combine plusieurs opérateurs, le compilateur applique un ordre de priorité (textitprecedence) strict. Il est impératif de connaître cet ordre pour éviter les erreurs logiques silencieuses.

Tableau : Ordre de priorité des opérateurs (du plus prioritaire au moins prioritaire)

Priorité	Opérateurs	Description
1	<code>()</code> , <code>.</code> , <code>[]</code>	Parenthèses, accès membre, indexeur
2	<code>!</code> , <code>++</code> , <code>--</code> , <code>(cast)</code>	Opérateurs unaires, négation, cast
3	<code>*</code> , <code>/</code> , <code>%</code>	Multiplication, division, modulo
4	<code>+</code> , <code>-</code>	Addition, soustraction
5	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>is</code> , <code>as</code>	Comparaison et vérification de type
6	<code>==</code> , <code>!=</code>	Égalité
7	<code>&&</code>	ET logique
8	<code> </code>	OU logique
9	<code>?:</code>	Ternaire conditionnel
10	<code>=</code> , <code>+=</code> , <code>-=</code> , etc.	Affectation

Règle d'ingénierie : Utilisez les parenthèses

Même si vous connaissez l'ordre de priorité, l'ajout de parenthèses explicites dans les expressions complexes est une pratique obligatoire en Clean Code. Elles rendent l'intention du développeur immédiatement lisible et préviennent les erreurs lors de la maintenance par un autre ingénieur.

5.9 Synthèse du Chapitre

Tableau : Récapitulatif des opérateurs critiques

Concept	Règle de l'Ingénierie
Division entière	<code>10 / 3</code> donne toujours <code>3</code> . Transtyper en <code>double</code> pour une précision flottante.
Court-circuit (<code>&&</code>, <code> </code>)	Optimise les performances CPU en annulant l'évaluation de la partie droite si le résultat final est déjà connu.
<code>is</code> et <code>as</code>	Outils indispensables pour la vérification dynamique et le transtypage sécurisé sans levée d'exceptions bloquantes.
Priorité	En cas de doute, encadrez avec des parenthèses explicites pour garantir l'intention.

6 Structures Conditionnelles

Les structures conditionnelles, ou structures de contrôle de flux, permettent de dévier l'exécution linéaire d'un programme en fonction de l'évaluation d'expressions booléennes (vrai ou faux). Ces structures constituent le cœur de la logique décisionnelle algorithmique.

6.1 L'instruction `if / else`

L'instruction `if` évalue une condition. Si celle-ci retourne `true`, le bloc de code associé est exécuté. Les clauses `else if` et `else` permettent de traiter des alternatives exclusives, optimisant le processus car l'évaluation s'arrête dès qu'une condition est remplie.

```

1 int debitReseau = 450; // Mbps
2
3 if (debitReseau >= 1000)
4 {
5     Console.WriteLine("Connexion Fibre Optique détectée.");
6 }
7 else if (debitReseau >= 100)
8 {
9     Console.WriteLine("Connexion Très Haut Débit (VDSL/Câble).");
10 }
11 else if (debitReseau >= 10)
12 {
13     Console.WriteLine("Connexion ADSL standard.");
14 }
15 else
16 {
17     Console.WriteLine("Connexion à débit réduit.");
18 }

```

Listing 29 – Logique décisionnelle en cascade

Règle d'ingénierie : Blocs d'instructions (Accolades)

Bien que le langage C# autorise l'omission des accolades `{}` si le bloc ne contient qu'une seule instruction, les conventions strictes (*Clean Code*) exigent de toujours les inclure. Cela prévient les bugs critiques de portée (*scope*) lors d'ajouts ultérieurs de code.

6.2 L'instruction `switch`

Le `switch` est utilisé pour évaluer une unique expression par rapport à une liste de modèles (*patterns*) ou de constantes. Il est structurellement plus lisible qu'une longue chaîne de `if/else` lors de la comparaison d'une même variable contre de multiples valeurs.

6.2.1 Syntaxe classique

Dans sa forme traditionnelle, chaque cas (`case`) doit se terminer obligatoirement par l'instruction `break` pour empêcher le compilateur de continuer l'exécution sur le cas suivant (erreur CS0163).

```

1 int codeErreurHttp = 404;
2
3 switch (codeErreurHttp)
4 {
5     case 200:
6         Console.WriteLine("Succès : La requête a abouti.");

```

```

7         break;
8     case 401:
9     case 403: // Regroupement de cas (Fall-through explicite)
10        Console.WriteLine("Erreur de sécurité : Accès refusé.");
11        break;
12    case 404:
13        Console.WriteLine("Erreur de routage : Ressource introuvable.");
14        break;
15    case 500:
16        Console.WriteLine("Erreur interne du serveur.");
17        break;
18    default:
19        Console.WriteLine("Code d'état non répertorié.");
20        break; // Le break est exigé même sur le cas par défaut
21 }

```

Listing 30 – Routage par instruction switch

6.2.2 Les Expressions switch (C# 8.0+)

Le C# moderne a introduit les **expressions switch**, qui transforment la structure de contrôle classique en une instruction retournant directement une valeur. La syntaxe remplace **case** par l'opérateur "flèche" => et **default** par le rejet (*discard*) _.

```

1 string protocole = "HTTPS";
2
3 // Retourne directement la valeur affectée au port
4 int portReseau = protocole switch
5 {
6     "HTTP"    => 80,
7     "HTTPS"   => 443,
8     "FTP"     => 21,
9     "SSH"     => 22,
10    _         => throw new ArgumentException("Protocole inconnu.")
11 };
12
13 Console.WriteLine($"Connexion établie sur le port {portReseau}");

```

Listing 31 – Affectation via expression switch moderne

6.3 L'opérateur Ternaire Conditionnel (?:)

L'opérateur ternaire condense une affectation conditionnelle simple sur une seule ligne de code. Il évalue une expression booléenne, puis retourne l'une des deux valeurs proposées.

Syntaxe : condition ? valeur_si_vrai : valeur_si_faux;

```

1 int chargeProcesseur = 85;
2
3 // Syntaxe classique en 6 lignes
4 string etatServeur;
5 if (chargeProcesseur > 90)
6 {
7     etatServeur = "CRITIQUE";
8 }
9 else
10 {
11     etatServeur = "STABLE";
12 }

```

```

13
14 // Syntaxe ternaire optimisée en 1 ligne
15 string etatServeurOptimise = (chargeProcesseur > 90) ? "CRITIQUE" : "
    STABLE";

```

Listing 32 – Optimisation par opérateur ternaire

Anti-pattern : Les ternaires imbriqués

L'imbrication d'opérateurs ternaires (ex : `a ? b : c ? d : e`) détruit la lisibilité du code. L'ingénierie logicielle proscrit formellement cette pratique au profit d'un `if / else` standard ou d'une expression `switch`.

6.4 Synthèse du Chapitre

Tableau : Rétrospective des structures de contrôle

Structure	Cas d'usage architectural
<code>if / else if</code>	Évaluation de conditions hétérogènes complexes (ex : plages de valeurs, combinaison de variables).
<code>switch classique</code>	Exécution de blocs d'instructions distincts basés sur l'état d'une variable unique.
<code>switch expression</code>	Affectation directe et fonctionnelle d'une variable basée sur de multiples valeurs statiques.
Ternaire (<code>?:</code>)	Affectation binaire rapide sur une seule ligne (à condition stricte de non-imbrication).

7 Les Boucles (Structures Itératives)

Les boucles, ou structures itératives, sont des mécanismes de contrôle de flux permettant d'exécuter un bloc d'instructions de manière répétée tant qu'une condition booléenne demeure évaluée à **true**. Elles sont indispensables en ingénierie logicielle pour l'automatisation de tâches répétitives, le parcours de structures de données ou l'attente d'événements asynchrones.

7.1 La boucle for

La boucle **for** est privilégiée lorsque le nombre d'itérations est connu à l'avance ou déterminé par une borne fixe. Elle encapsule l'initialisation de l'index, la condition de maintien et l'incréméntation (ou décréméntation) dans une seule ligne de déclaration.

```
1 for (int i = 0; i < 5; i++)
2 {
3     Console.WriteLine($"Itération numéro : {i}");
4 }
```

Analyse de l'exécution :

- **Initialisation** (`int i = 0`) : Exécutée une seule fois au démarrage de la boucle.
- **Condition** (`i < 5`) : Évaluée avant chaque itération. Si elle est **false**, la boucle s'arrête et le programme continue à l'instruction suivante.
- **Incréméntation** (`i++`) : Exécutée à la fin de chaque itération, juste avant de réévaluer la condition.

7.2 La boucle while

La boucle **while** évalue sa condition de maintien *avant* d'exécuter son bloc d'instructions. Elle est typiquement employée lorsque le nombre d'itérations est indéterminé, dépendant d'un état dynamique du système ou de la lecture d'un flux de données (ex : lecture de fichiers, écoute réseau).

```
1 int bufferSize = 0;
2 while (bufferSize < 5)
3 {
4     Console.WriteLine($"Traitement du bloc mémoire {bufferSize}...");
5     bufferSize++;
6 }
```

Contrairement au **for**, l'initialisation de la variable de contrôle est externe à la boucle, et son altération doit obligatoirement figurer à l'intérieur du bloc pour éviter une boucle infinie.

7.3 La boucle do...while

Contrairement à la boucle **while**, la variante **do...while** évalue la condition *après* l'exécution du bloc. Cela garantit que les instructions internes seront exécutées au strict minimum une fois, indépendamment de l'état initial.

```
1 int retryCount = 0;
2 do
3 {
4     Console.WriteLine($"Tentative de connexion réseau (Essai {retryCount + 1})");
5     retryCount++;
6 } while (retryCount < 5);
```

Cette boucle est très utile pour les scénarios nécessitant au moins une action initiale, tels que les tentatives de connexion (*retry policy*) ou l'exécution d'un menu interactif.

7.4 La boucle foreach

La structure **foreach** offre une syntaxe abstraite et sécurisée pour itérer sur l'ensemble des éléments d'une collection (tableaux, listes) implémentant l'interface **IEnumerable**. Elle élimine la gestion manuelle de l'index et prévient les erreurs de dépassement.

```
1 int[] tcpPorts = { 80, 443, 8080, 1433 };
2 foreach (int port in tcpPorts)
3 {
4     Console.WriteLine($"Analyse du port : {port}");
5 }
```

Bien que cette boucle produise un code plus concis et robuste, la variable d'itération (**port** dans l'exemple) est en lecture seule. Vous ne pouvez pas modifier directement l'élément du tableau via cette variable.

7.5 Contrôle de Flux : break et continue

Au sein de toute structure itérative, il est possible d'altérer le flux standard à l'aide de deux mots-clés :

- **break** : Interrompt immédiatement et définitivement la boucle englobante, forçant le programme à passer à l'instruction qui suit la boucle.
- **continue** : Interrompt l'itération courante et passe directement à l'évaluation de la condition pour démarrer l'itération suivante.

Pièges Courants & Erreurs

- **La boucle infinie** : Oublier d'incrémenter la variable d'arrêt ou de modifier la condition dans une boucle **while**, entraînant le blocage complet du thread (*Thread Starvation*).
- **Modification de la collection** : Tenter d'ajouter ou de supprimer un élément d'une liste qui est actuellement en train d'être parcourue par un **foreach** déclenche inmanquablement une **InvalidOperationException** à l'exécution.
- **Erreur de borne (*Off-by-one error*)** : Utiliser **<=** au lieu de **<** lors du parcours d'un tableau classique avec une boucle **for**. Puisque les tableaux sont indexés à partir de 0, cela mène à une **IndexOutOfRangeException**.

7.6 Synthèse

- **for** : À privilégier pour les itérations indexées dont le nombre est prévisible et fini.
- **while** : Idéal pour les processus en attente d'un état ou lisant un flux de données continu.
- **do...while** : Utile lorsqu'un premier traitement est impératif avant toute vérification de condition.
- **foreach** : La méthode la plus élégante et robuste pour parcourir des structures de données en toute sécurité.

8 Tableaux et Collections de base

En ingénierie logicielle, la capacité à regrouper et manipuler efficacement de multiples données connexes est primordiale. C# propose plusieurs structures fondamentales : depuis les tableaux primitifs à taille fixe jusqu'aux collections génériques dynamiques fournies par le framework .NET. Le choix de la bonne structure influence directement les performances en termes d'allocation mémoire et de rapidité d'exécution.

8.1 Les Tableaux (Array)

Les tableaux représentent la structure la plus bas niveau du langage. Ils stockent des éléments de même type de manière contiguë en mémoire. Leur caractéristique principale est leur **taille fixe**, définie obligatoirement au moment de l'instanciation.

```

1 // Déclaration et instanciation d'un tableau de 5 entiers
2 int[] tableau = new int[5];
3
4 // Affectation par index (zéro-indexé)
5 tableau[0] = 10;
6 tableau[1] = 20;
7
8 // Initialisation directe (sucre syntaxique)
9 string[] jours = { "Lundi", "Mardi", "Mercredi" };
10
11 // Redimensionnement dynamique
12 Array.Resize(ref tableau, 10);

```

L'accès à un élément par son index s'effectue en temps constant $O(1)$. Cependant, la méthode `Array.Resize` crée en réalité un *nouveau* tableau en mémoire et y copie tous les éléments de l'ancien. Cette opération est très coûteuse en performance, c'est pourquoi les tableaux ne doivent être utilisés que lorsque la quantité d'éléments est connue et définitive.

8.2 Les Listes (List<T>)

Pour pallier la rigidité des tableaux, le framework .NET fournit la collection générique `List<T>`. Elle encapsule un tableau sous-jacent et gère automatiquement et silencieusement son redimensionnement lors de l'ajout ou de la suppression d'éléments.

```

1 using System.Collections.Generic;
2
3 List<int> liste = new List<int>();
4
5 // Ajout dynamique à la fin de la collection
6 liste.Add(10);
7 liste.Add(20);
8 liste.Add(30);
9
10 // Suppression de la première occurrence de la valeur 20
11 liste.Remove(20);
12
13 // Accès par index et parcours
14 Console.WriteLine(liste[0]); // Affiche 10

```

La `List<T>` est la structure par défaut recommandée en C# pour stocker des données séquentielles nécessitant de la flexibilité.

8.3 Les Dictionnaires (Dictionary<TKey, TValue>)

Le dictionnaire implémente une table de hachage (*Hash Table*). Il stocke des données sous forme de paires "Clé/Valeur". La recherche d'une valeur à partir de sa clé s'effectue via un calcul de hachage, garantissant un temps de recherche proche de $O(1)$.

```
1 using System.Collections.Generic;
2
3 // Déclaration : Clés de type string, Valeurs de type int
4 Dictionary<string, int> ages = new Dictionary<string, int>();
5
6 // Ajout de paires (les clés doivent être uniques)
7 ages.Add("Alice", 28);
8 ages.Add("Bob", 35);
9
10 // Suppression via la clé unique
11 ages.Remove("Alice");
12
13 // Accès très rapide par clé
14 Console.WriteLine(ages["Bob"]); // Affiche 35
```

Le dictionnaire est incontournable lorsque les données doivent être indexées et retrouvées via un identifiant métier (ex : un matricule employé, un code produit) plutôt que par une simple position ordinale.

8.4 Les Énumérations (enum)

Bien qu'il ne s'agisse pas d'une collection à proprement parler, l'**enum** est un type fondamental permettant de définir un ensemble de constantes liées. Il améliore drastiquement la lisibilité du code en remplaçant les entiers bruts (souvent qualifiés de "nombres magiques") par des identifiants sémantiques clairs.

```
1 public enum StatutCommande
2 {
3     EnAttente = 0,
4     Payee = 1,
5     Expediee = 2,
6     Livree = 3
7 }
8
9 // Utilisation typée et robuste
10 StatutCommande statutActuel = StatutCommande.Payee;
11
12 if (statutActuel == StatutCommande.Payee)
13 {
14     Console.WriteLine("La commande peut être expédiée.");
15 }
```

Pièges Courants & Erreurs

- **IndexOutOfRangeException** : Essayer d'accéder à `tableau[5]` sur un tableau de taille 5 (les index valides vont de 0 à 4 puisque indexés à partir de 0).
- **KeyNotFoundException** : Tenter de lire une valeur dans un dictionnaire avec une clé inexistante (ex : `ages["Inconnu"]`). Si l'existence de la clé est incertaine, utilisez plutôt `ages.TryGetValue(...)`.
- **ArgumentException (Clé doublée)** : Utiliser la méthode `Add` d'un dictionnaire avec une clé qui y est déjà présente. Un dictionnaire exige des clés strictement uniques.

8.5 Fonctions de Manipulation Avancées

Le framework .NET propose de nombreuses fonctions utilitaires pour interagir avec les collections sans réinventer la roue.

8.5.1 Utilitaires pour Tableaux et Listes

```
1 int[] nombres = { 5, 3, 1, 4, 2 };
2
3 // Tri et Inversion
4 Array.Sort(nombres); // { 1, 2, 3, 4, 5 }
5 Array.Reverse(nombres); // { 5, 4, 3, 2, 1 }
6
7 // Recherche d'éléments
8 bool contientTrois = Array.Exists(nombres, n => n == 3);
9 int premierPair = Array.Find(nombres, n => n % 2 == 0); // 4
10 int[] tousPairs = Array.FindAll(nombres, n => n % 2 == 0); // { 4, 2 }
11
12 // Effacement et Copie
13 Array.Clear(nombres, 0, nombres.Length); // Rempli de zéros
```

8.5.2 Utilitaires pour Dictionnaires

```
1 Dictionary<string, int> inventaire = new Dictionary<string, int>
2 {
3     { "Pommes", 50 },
4     { "Bananes", 20 }
5 };
6
7 // Vérification d'existence (O(1))
8 bool aPommes = inventaire.ContainsKey("Pommes"); // true
9 bool aQuantiteVingt = inventaire.ContainsValue(20); // true
10
11 // Extraction sécurisée (pattern TryGetValue)
12 if (inventaire.TryGetValue("Oranges", out int quantite))
13 {
14     Console.WriteLine($"Oranges en stock : {quantite}");
15 }
16 else
17 {
18     Console.WriteLine("Oranges introuvables.");
19 }
```

8.6 Synthèse

- **Tableau** (`T[]`) : Taille fixe, bas niveau, performant. Idéal pour les données immuables.
- **Liste** (`List<T>`) : Taille dynamique, encapsulation intelligente. Idéale pour les collections appelées à évoluer en nombre.
- **Dictionnaire** (`Dictionary<K, V>`) : Recherche instantanée par clé métier. L'arme absolue pour retrouver une donnée précise sans parcourir toute la collection.
- **Énumération** (`enum`) : Permet de définir des états restreints de manière propre, typée et compréhensible.

9 Fonctions et Méthodes

En ingénierie logicielle, la création de méthodes (le terme C# pour désigner les fonctions rattachées à une classe) est fondamentale pour garantir la modularité, la réutilisabilité et la lisibilité du code. Une méthode regroupe un ensemble d'instructions accomplissant une tâche algorithmique ou métier spécifique.

9.1 Déclaration, Paramètres et Valeur de Retour

La signature d'une méthode définit son comportement externe : son niveau d'accès, son type de retour, son identifiant et ses paramètres d'entrée. Si la méthode ne produit aucune donnée de sortie exploitable, son type de retour est déclaré comme `void`.

```

1 // Méthode 'void' (aucune valeur retournée)
2 public void EcrireLog(string message)
3 {
4     Console.WriteLine($"[LOG] {DateTime.UtcNow} : {message}");
5 }
6
7 // Méthode avec retour d'une donnée typée (decimal)
8 public decimal CalculerTTC(decimal montantHT, decimal tauxTVA)
9 {
10     decimal montantTVA = montantHT * (tauxTVA / 100);
11     return montantHT + montantTVA;
12 }
```

La clause `return` stoppe immédiatement l'exécution de la méthode et restitue la valeur à l'appelant. Le type de cette valeur doit correspondre strictement à la signature définie.

9.2 Paramètres Optionnels et Surcharge

C# offre deux mécanismes distincts pour assouplir l'appel des méthodes :

9.2.1 Valeurs par défaut (Paramètres Optionnels)

Un paramètre peut se voir attribuer une valeur par défaut dans la signature. Il devient alors optionnel lors de l'appel. Les paramètres optionnels doivent obligatoirement être déclarés *à la fin* de la liste des paramètres.

```

1 public void ConfigurerConnexion(string ip, int port = 1433, int timeout
2     = 30)
3 {
4     Console.WriteLine($"Connexion à {ip}:{port} (Timeout: {timeout}s)");
5 }
6
7 // Appels valides :
8 ConfigurerConnexion("192.168.1.10"); // port = 1433, timeout = 30
9 ConfigurerConnexion("10.0.0.5", 8080); // timeout = 30
```

9.2.2 Surcharge de Méthodes (*Overloading*)

La surcharge consiste à définir plusieurs méthodes portant **le même nom**, mais ayant une **signature différente** (nombre, type ou ordre des paramètres). Le compilateur détermine la méthode à exécuter en analysant les arguments passés.

L'avantage principal de l'*overloading* est de proposer une API naturelle. Au lieu d'inventer des noms comme `AfficherProfilBasique` et `AfficherProfilComple`t, on centralise l'action sous un seul verbe.

```

1 // Version 1 : 1 paramètre (string)
2 public void AfficherProfil(string pseudo)
3 {
4     Console.WriteLine($"Profil basique : {pseudo}");
5 }
6
7 // Version 2 : 2 paramètres (string, int) -> Le nombre change
8 public void AfficherProfil(string pseudo, int age)
9 {
10     Console.WriteLine($"Profil complet : {pseudo}, {age} ans");
11 }
12
13 // Version 3 : 2 paramètres (int, string) -> L'ordre change (signature
    valide)
14 public void AfficherProfil(int age, string pseudo)
15 {
16     Console.WriteLine($"[Mode inversé] : {pseudo}");
17 }

```

9.3 Nombre Variable d'Arguments (params)

Le mot-clé **params** permet à une méthode d'accepter un nombre indéfini d'arguments du même type, qui seront encapsulés automatiquement dans un tableau. Ce mot-clé ne peut être appliqué qu'au tout dernier paramètre de la signature.

```

1 public decimal CalculerMoyenne(params decimal[] valeurs)
2 {
3     if (valeurs.Length == 0) return 0;
4
5     decimal somme = 0;
6     foreach (decimal val in valeurs)
7     {
8         somme += val;
9     }
10    return somme / valeurs.Length;
11 }
12
13 // L'appelant peut passer une liste d'arguments séparés par des virgules
14 decimal moyenne = CalculerMoyenne(12.5m, 15.0m, 18.5m);

```

9.4 Passage de Paramètres : ref et out

Par défaut, le passage d'arguments (pour les types valeur) s'effectue **par valeur** (copie isolée). Le modificateur **ref** exige un passage par référence (adresse mémoire), permettant à la méthode de muter directement la variable de l'appelant. La variable doit obligatoirement être initialisée avant l'appel.

Le modificateur **out** opère de la même manière, mais son rôle est d'exporter un résultat. Il exempte l'appelant d'initialiser la variable, mais force contractuellement la méthode à lui assigner une valeur avant l'instruction de retour.

```

1 // Exemple d'utilisation du pattern TryParse avec 'out' inline
2 string fluxReseau = "1024";
3
4 if (int.TryParse(fluxReseau, out int portServeur))
5 {
6     Console.WriteLine($"Port alloué : {portServeur}");
7 }

```

*Note : Bien que `out` fut longtemps utilisé pour retourner de multiples valeurs, l'architecture C# moderne préconise plutôt l'utilisation des **Tuples** pour cet usage.*

9.5 Méthodes Statiques vs d'Instance

- **Méthodes d'instance** : Nécessitent l'instanciation préalable de la classe (la création d'un objet en mémoire). Elles peuvent manipuler l'état interne (les champs) propre à cette instance spécifique.
- **Méthodes statiques (`static`)** : Sont attachées au type lui-même (la classe) et non à une instance. Elles ne peuvent pas accéder aux données d'une instance. Elles sont conçues pour des opérations utilitaires pures (ex : `Math.Abs()` ou `int.TryParse()`).

Pièges Courants & Erreurs

- **Ambiguïté de surcharge** : Créer deux méthodes avec des signatures identiques mais des types de retour différents génère une erreur de compilation, car le type de retour ne fait pas partie de la signature permettant de résoudre la surcharge.
- **Rupture du contrat `out`** : CS0177 survient inévitablement si la méthode omet d'affecter une valeur au paramètre `out` avant de se terminer.
- **Ordre des paramètres optionnels** : Tenter de définir un paramètre optionnel *avant* un paramètre requis provoquera une erreur stricte du compilateur.

9.6 Synthèse

- **`void` / Type de retour** : Définit si la méthode opère comme une action silencieuse ou comme une fonction calculant un résultat.
- **Surcharge & Paramètres optionnels** : Flexibilisent l'API de votre classe et facilitent l'appel de vos méthodes.
- **`params`** : Simplifie le passage d'une liste dynamique de variables du même type.
- **`ref` / `out`** : Permettent le passage par référence, utiles pour muter la variable appelante ou implémenter le pattern `TryParse`.
- **`static`** : Rend la méthode globale et indépendante de tout état d'instance.

10 Manipulation des Types de Base (Chaînes et Nombres)

En C#, le type `string` (alias de la classe `System.String`) représente une séquence de caractères Unicode. Une caractéristique fondamentale à assimiler en ingénierie logicielle est l'**immutabilité** des objets `string` : une fois créée en mémoire, une chaîne de caractères ne peut plus être modifiée. Toute opération de transformation (mise en majuscules, remplacement, découpage) instancie et retourne une *nouvelle* chaîne en mémoire, laissant la chaîne d'origine intacte.

10.1 Propriétés et Méthodes de Base

```

1 string texte = "    Connexion au serveur...    ";
2
3 // Propriété Length : Obtient la longueur totale (incluant les espaces)
4 int longueur = texte.Length; // 29
5
6 // Trim() : Supprime les espaces blancs (espaces, tabulations, sauts de
7 // ligne) aux extrémités
8 string clean = texte.Trim(); // "Connexion au serveur..."
9
10 // Transformation de la casse
11 string majuscule = clean.ToUpper(); // "CONNEXION AU SERVEUR..."
12 string minuscule = clean.ToLower(); // "connexion au serveur..."

```

10.2 Recherche et Vérification

Pour vérifier le contenu d'une chaîne sans nécessairement l'extraire, C# propose des méthodes d'évaluation retournant un booléen ou un index.

```

1 string log = "[ERROR] Timeout de la base de données";
2
3 // Vérifications booléennes
4 bool estErreur = log.StartsWith("[ERROR]"); // true
5 bool concerneBDD = log.EndsWith("données"); // true
6 bool contientTimeout = log.Contains("Timeout"); // true
7
8 // Recherche d'index (retourne la position zéro-indexée, ou -1 si
9 // introuvable)
10 int position = log.IndexOf("Timeout"); // 8

```

10.3 Découpage, Concaténation et Remplacement

La manipulation de données brutes (comme des fichiers CSV ou des logs) requiert souvent d'extraire, d'isoler ou de remplacer des segments de texte.

```

1 // Substring(startIndex, length) : Extrait un segment précis
2 string identifiant = "USER_1024_ADMIN";
3 string idNum = identifiant.Substring(5, 4); // Extrait "1024"
4
5 // Replace(oldValue, newValue) : Remplace toutes les occurrences
6 string requeteSql = "SELECT * FROM [Table]";
7 string requeteSecurisee = requeteSql.Replace("[Table]", "Utilisateurs");
8
9 // Split() : Divise une chaîne en tableau (ex: lecture de CSV)
10 string ligneCsv = "Alice,Dupont,30,Paris";
11 string[] colonnes = ligneCsv.Split(',');
12 // colonnes = ["Alice", "Dupont", "30", "Paris"]

```

```

13
14 // Join() : Recompose un tableau en une chaîne avec un séparateur
15 string nouvelleLigne = string.Join(";", colonnes);
16 // "Alice;Dupont;30;Paris"

```

10.4 Insertion et Suppression

```

1 string chemin = "C:\\dossier\\fichier.txt";
2
3 // Insert(index, valeur) : Insère une chaîne à une position précise
4 string cheminBackup = chemin.Insert(18, "_backup");
5 // "C:\\dossier\\fichier_backup.txt"
6
7 // Remove(index, length) : Supprime une partie de la chaîne
8 string nomCourt = cheminBackup.Remove(0, 11);
9 // "fichier_backup.txt"

```

10.5 Interpolation et Optimisation (StringBuilder)

10.5.1 Interpolation de chaînes (\$"")

L'interpolation, introduite en C# 6, permet d'injecter directement des expressions ou variables dans une chaîne à l'aide d'accolades {}. C'est l'approche la plus lisible pour la construction de chaînes dynamiques, bien supérieure à la concaténation par l'opérateur +.

```

1 string serveur = "192.168.1.10";
2 int port = 443;
3 // Le préfixe $ active l'interpolation
4 string uri = $"https://{serveur}:{port}/api/v1/status";

```

10.5.2 Optimisation avec StringBuilder

À cause de l'immuabilité du type `string`, une boucle qui concatène des milliers de chaînes via += provoquera autant d'allocations mémoires superflues, saturant le *Garbage Collector*. Dans les contextes de forte performance ou de modifications intensives, il est impératif d'utiliser la classe `StringBuilder` (namespace `System.Text`).

```

1 using System.Text;
2
3 StringBuilder sb = new StringBuilder();
4 for (int i = 0; i < 10000; i++)
5 {
6     // Ne recrée pas de nouvel objet à chaque tour, modifie le buffer
    existant
7     sb.Append($"Ligne {i} | ");
8 }
9 string resultatFinal = sb.ToString();

```


Pièges Courants & Erreurs

- **Immutabilité ignorée** : Écrire `texte.Trim()`; seul sans récupérer le résultat (`texte = texte.Trim();`) n'aura aucun effet sur la variable `texte`.
- **NullReferenceException** : Appeler `Contains` ou `ToLower` sur une variable `string` valant `null` provoquera un crash instantané. Il faut toujours vérifier la nullité avec `string.IsNullOrEmpty(texte)` ou `string.IsNullOrWhiteSpace(texte)` avant manipulation.
- **Concaténation dans des boucles** : Utiliser l'opérateur `+=` sur une `string` dans une boucle de plus d'une dizaine d'itérations est considéré comme un défaut d'architecture (*Code Smell*) pour les performances mémoire.

10.6 Manipulation des Nombres et Mathématiques

La classe statique `System.Math` fournit l'ensemble des opérations mathématiques fondamentales nécessaires en ingénierie logicielle. Parallèlement, les types primitifs numériques disposent de méthodes de parsing sécurisées.

```
1 // Méthodes mathématiques de base
2 int absolu = Math.Abs(-5); // 5
3 int maximum = Math.Max(10, 20); // 20
4 double puissance = Math.Pow(2, 3); // 8 (2 au cube)
5 double arrondi = Math.Round(5.5); // 6
6
7 // Limitation d'une valeur dans un intervalle (Clamp)
8 int valeurRestreinte = Math.Clamp(15, 0, 10); // 10
```

10.6.1 Conversion et Parsing

La conversion d'une chaîne de caractères vers un entier est une opération risquée qui doit toujours être sécurisée.

```
1 string saisie = "42";
2
3 // Parse : Lève une exception (FormatException) si la chaîne est
4 // invalide
5 int nombre = int.Parse(saisie);
6
7 // TryParse : Approche sécurisée (ne lève pas d'exception, retourne un
8 // booléen)
9 if (int.TryParse("texte_invalide", out int resultatSecurise))
10 {
11     Console.WriteLine($"Conversion réussie : {resultatSecurise}");
12 }
13 else
14 {
15     Console.WriteLine("Échec de la conversion.");
16 }
```

10.7 Synthèse

- **Propriétés fondamentales** : Les objets `string` sont immuables. Chaque modification génère une nouvelle allocation mémoire.
- **Évaluation** : Utiliser `StartsWith`, `Contains` et `IndexOf` pour analyser le contenu sans créer de nouvelles chaînes.

- **Formatage dynamique** : Privilégier systématiquement l'interpolation (`$""`) pour la lisibilité du code.
- **Performance** : Utiliser exclusivement `StringBuilder` lors de la construction itérative de chaînes dans des boucles.

11 Gestion des Erreurs et Exceptions

En ingénierie logicielle, un programme informatique ne s'exécute jamais dans un environnement parfait. Des anomalies externes (perte de réseau, fichier inexistant, base de données injoignable) ou internes (division par zéro, référence nulle) peuvent survenir. Le framework .NET gère ces anomalies via le concept d'**Exception**.

Une exception est un objet (héritant de la classe de base `System.Exception`) qui est "levé" (*thrown*) par le runtime lorsqu'une erreur critique empêche la poursuite normale de l'exécution. Si cette exception n'est pas "capturée" (*caught*) par le développeur, le programme s'arrête brutalement (*Crash*).

11.1 Le Bloc `try / catch / finally`

L'interception des erreurs s'effectue via une structure de contrôle dédiée.

```

1 try
2 {
3     // 1. Zone à risque : code susceptible de déclencher une erreur
4     int diviseur = 0;
5     int resultat = 10 / diviseur;
6
7     // Si une erreur survient au-dessus, les lignes suivantes sont ignor
   ées
8     Console.WriteLine("Calcul terminé.");
9 }
10 catch (DivideByZeroException ex)
11 {
12     // 2. Interception spécifique d'une division par zéro
13     Console.WriteLine($"[CRITIQUE] Tentative de division par zéro : {ex.
   Message}");
14 }
15 catch (Exception ex)
16 {
17     // 3. Interception globale (Fallback) pour toute autre erreur non pr
   évue
18     Console.WriteLine($"[ERREUR INCONNUE] {ex.Message}");
19 }
20 finally
21 {
22     // 4. Nettoyage : s'exécute TOUJOURS, qu'il y ait eu une erreur ou
   non
23     Console.WriteLine("Libération de la mémoire de calcul.");
24 }
```

Il est primordial en C# de toujours ordonner les blocs `catch` du type d'exception le plus précis (`ex : DivideByZeroException`) vers le type le plus générique (`Exception`).

11.1.1 Exceptions courantes du Framework

- `NullReferenceException` : Tentative d'accès à un membre d'un objet valant `null`.
- `IndexOutOfRangeException` : Accès à un index de tableau inexistant.
- `FormatException` : Échec lors du *parsing* d'un type (`ex : int.Parse("ABC")`).

11.2 Le Mot-Clé `throw` et la Propagation

Il est souvent nécessaire de déclencher volontairement une erreur lorsqu'une validation métier échoue. Le mot-clé `throw` interrompt immédiatement la méthode courante et remonte l'exception

à l'appelant.

```
1 public void TraiterPaieement(decimal montant)
2 {
3     if (montant <= 0)
4     {
5         // Levée intentionnelle d'une exception
6         throw new ArgumentOutOfRangeException(nameof(montant), "Le
montant doit être positif.");
7     }
8
9     // Suite du traitement...
10 }
```

11.3 Création d'Exceptions Personnalisées

Pour modéliser des erreurs métiers spécifiques (ex : un solde insuffisant), il convient de créer ses propres classes d'exception héritant de `Exception`.

```
1 // Définition d'une exception métier
2 public class SoldeInsuffisantException : Exception
3 {
4     public SoldeInsuffisantException(string message) : base(message) { }
5 }
6
7 // Utilisation
8 if (soldeCompte < prixPanier)
9 {
10     throw new SoldeInsuffisantException($"Fonds manquants. Solde: {
soldeCompte}");
11 }
```

11.4 Gestion des Ressources : Le Bloc using

La libération des ressources dites "non-managées" (fichiers système, connexions réseaux, flux de base de données) ne doit pas dépendre du Garbage Collector. Le bloc `using` garantit que la méthode `Dispose()` de la ressource sera appelée à la fin du bloc, **même si une exception est levée** à l'intérieur.

```
1 // La "using declaration" (sans accolades) assure la fermeture à la fin
de la méthode
2 using var reader = new StreamReader("C:\\data\\config.json");
3 string contenu = reader.ReadToEnd();
4 Console.WriteLine(contenu);
5 // reader.Dispose() est appelé implicitement à la sortie de la méthode
courante
```

Pièges Courants & Anti-Patterns

- **Le Catch Vide (*Swallowing*)** : Écrire un `catch(Exception) { }` sans logger ou traiter l'erreur masque silencieusement les bugs critiques du programme.
- **Mauvaise Propagation (`throw ex;`)** : Écrire `throw ex;` dans un bloc `catch` écrase la trace d'appel (*Stack Trace*) d'origine. Il faut utiliser le mot-clé `throw;` seul pour propager l'erreur sans la masquer.
- **Utiliser les exceptions pour le contrôle de flux** : Les exceptions coûtent extrêmement cher en temps de calcul CPU. Ne les utilisez jamais pour vérifier des conditions prévisibles (préférez `int.TryParse` plutôt qu'un `try/catch` sur `int.Parse`).

11.5 Synthèse

- `try` : Encapsule le code susceptible d'échouer.
- `catch` : Intercepte et traite un type spécifique d'erreur.
- `finally` : Assure l'exécution du code de nettoyage critique.
- `throw` : Déclenche manuellement ou propage une exception.
- `using` : Remplace avantageusement un bloc `try/finally` pour garantir la destruction des ressources non-managées (`IDisposable`).

12 Programmation Orientée Objet — Bases

12.1 Introduction Théorique

La Programmation Orientée Objet (POO) est un paradigme d'ingénierie logicielle structurant l'architecture d'une application autour d'entités cohésives appelées **objets**. Contrairement à la programmation procédurale qui sépare chronologiquement la donnée de la logique de traitement, la POO encapsule l'état (les champs ou propriétés) et le comportement (les méthodes) au sein d'une même structure mémoire.

Cette approche permet de concevoir des systèmes complexes en favorisant la modularité, la réutilisabilité du code et le masquage de l'implémentation interne (encapsulation). En C#, tout le framework .NET repose intrinsèquement sur ce paradigme, depuis les types primitifs jusqu'aux composants de haut niveau.

12.2 Syntaxe et Règles : Classes, Objets et Types

Une **classe** définit le contrat, la structure mémoire et le comportement d'un type. Elle agit comme un modèle d'allocation. Un **objet** est une instance de cette classe, allouée dynamiquement en mémoire lors de l'exécution, via l'opérateur `new`.

12.2.1 Types Valeur (`struct`) vs Types Référence (`class`)

Le C# distingue deux grandes familles d'allocation mémoire pour les structures de données personnalisées :

- **class (Type référence)** : Les instances sont allouées sur le **tas** (Heap). La variable contient uniquement un pointeur (une référence) vers l'adresse mémoire de l'objet. Adapté pour les entités métier complexes, les services et le partage d'état.
- **struct (Type valeur)** : Les instances sont allouées directement sur la **pile** (Stack) ou encapsulées en ligne (`inline`) dans un autre type. La variable contient directement les données.

Idéal pour les petites structures de données immuables (comme des coordonnées géospatiales ou des vecteurs mathématiques) pour limiter la pression sur le Garbage Collector (GC).

12.2.2 Encapsulation et Modificateurs d'Accès

L'encapsulation restreint la visibilité de l'état interne d'un composant. Le C# utilise des modificateurs d'accès pour définir ce contrat d'interface :

- **public** : Accessible par n'importe quel code appelant.
- **private** : Accessible uniquement à l'intérieur du bloc de la classe (visibilité par défaut).
- **protected** : Accessible dans la classe et dans ses futures classes dérivées (héritage).
- **internal** : Accessible uniquement au sein du même assemblage (le même projet ou la même DLL).

12.2.3 Constructeurs et le mot-clé `this`

Le constructeur est la méthode d'initialisation appelée automatiquement lors de l'allocation d'un objet. Le mot-clé `this` fait référence à l'instance courante de l'objet en cours de traitement, permettant de lever les ambiguïtés de nommage (masquage/shadowing) entre les paramètres du constructeur et les champs propres à la classe.

12.2.4 Propriétés (`get`, `set`, `init`)

Les propriétés exposent des méthodes d'accès (accesseurs) dissimulées derrière une syntaxe d'affectation de variable, offrant un contrôle strict sur la lecture et l'écriture des données encapsulées. Le modificateur `init` (introduit avec C# 9) restreint l'écriture de la propriété exclusivement à la phase d'initialisation de l'objet, garantissant par la suite l'immuabilité absolue de la donnée.

12.3 Exemple de Code Commenté

L'exemple suivant illustre la création d'un composant de gestion de connexion réseau, démontrant l'encapsulation, la validation des données et l'initialisation sécurisée.

```
1 public class NetworkConnection
2 {
3     // 1. Champ privé (état interne encapsulé)
4     private int _timeoutMs;
5
6     // 2. Propriété en lecture publique, écriture limitée à l'
7     //    initialisation
8     public string HostIp { get; init; }
9
10    // 3. Propriété avec logique métier dans le mutateur (set)
11    public int Timeout
12    {
13        get { return _timeoutMs; }
14        set
15        {
16            if (value < 0)
17                throw new ArgumentOutOfRangeException(nameof(value), "Le
18                timeout doit être positif.");
19            _timeoutMs = value;
20        }
21    }
22 }
```

```

21
22 // 4. Constructeur avec utilisation de 'this' pour lever l'ambiguïté
23 public NetworkConnection(string hostIp, int timeoutMs)
24 {
25     this.HostIp = hostIp;
26     this.Timeout = timeoutMs; // Appelle le 'set' de la propriété (
qui valide la donnée)
27 }
28
29 // 5. Méthode d'instance
30 public void OpenConnection()
31 {
32     Console.WriteLine($"Connexion à {HostIp} avec un délai de {
Timeout}ms...");
33 }
34 }
35
36 class Program
37 {
38     static void Main()
39     {
40         // 6. Instanciation via l'opérateur 'new'
41         NetworkConnection conn = new NetworkConnection("192.168.1.10",
5000);
42
43         conn.Timeout = 2000; // Modification valide via la propriété
// conn.HostIp = "10.0.0.1"; // ERREUR DE COMPILATION : La
propriété 'init' est immuable
44
45         conn.OpenConnection();
46     }
47 }
48

```

Listing 33 – Implémentation d'un composant de connexion réseau

Explications ligne par ligne :

- **Ligne 4** : Déclaration d'un champ privé `_timeoutMs`. La convention standard de l'écosystème C# stipule de préfixer les champs d'instance privés par un *underscore* (`_`).
- **Ligne 7** : La propriété `HostIp` utilise `init`. Sa valeur ne peut être définie que dans le constructeur ou lors de l'initialisation de l'objet, empêchant toute mutation involontaire de l'adresse cible par la suite.
- **Lignes 10-20** : La propriété `Timeout` agit comme une façade sécurisée pour le champ interne `_timeoutMs`, en y injectant une règle de validation (le rejet immédiat des valeurs temporelles négatives).
- **Ligne 25** : Le mot-clé `this` spécifie explicitement que l'on assigne la valeur à la propriété `HostIp` de l'instance courante, et non au paramètre homonyme si celui-ci existait.
- **Ligne 41** : L'opérateur `new` alloue la mémoire requise sur le tas (Heap) pour une nouvelle instance de `NetworkConnection` et déclenche son constructeur.

Pièges courants & Erreurs de compilation

- `NullReferenceException` : Omettre d'utiliser l'opérateur `new`. Déclarer `NetworkConnection conn`; crée uniquement une référence vide pointant vers `null`. Tenter d'appeler `conn.OpenConnection()` dans cet état lèvera une exception fatale à l'exécution.
- **Erreur de visibilité (CS0122)** : Tenter d'accéder à un membre `private` (comme `_timeoutMs`) depuis l'extérieur de la classe (par exemple depuis `Program.Main`). L'état interne doit exclusivement être manipulé via des `public` propriétés ou méthodes.

Le Piège Silencieux des Exceptions d'Arguments

Il existe une **incohérence historique et asymétrique** dans l'API .NET :

- `new ArgumentException("Mon message")` attend le **message** en paramètre unique.
- `new ArgumentNullException("Mon message")` attend le **nom du paramètre** (`paramName`) en paramètre unique !

Si vous passez uniquement un message à `ArgumentNullException` ou `ArgumentOutOfRangeException`, le compilateur l'accepte, mais à l'exécution le texte sera assigné au nom du paramètre. Le vrai message affiché sera un message système absurde du type : *Value cannot be null. (Parameter 'Le flux est vide.')*. **Règle d'or** : Utilisez toujours la surcharge à deux paramètres (`nameof(param)`, `"Message"`) pour ces exceptions spécifiques.

12.4 Synthèse / À retenir

- **Classe vs Objet** : La classe est le plan d'architecture abstrait, l'objet est l'instance mémoire concrète.
- **class vs struct** : Les classes (`class`) sont passées par référence (Heap), les structures (`struct`) sont passées par valeur (Stack).
- **Encapsulation** : Protégez systématiquement l'état interne d'un objet avec `private` et exposez des accès contrôlés via des **propriétés**.
- **Immuabilité** : Privilégiez l'utilisation de `init` (plutôt que `set`) pour les données de configuration qui ne doivent pas être altérées après la phase d'instanciation.
- L'opérateur `new` est impératif pour allouer dynamiquement un objet de type référence avant d'utiliser ses méthodes.

13 POO — Héritage et Polymorphisme

13.1 L'Héritage et le Partage de Comportement

L'héritage permet de créer une hiérarchie de classes en factorisant la logique technique commune dans une classe de base (*Base Class*) et en spécialisant cette logique dans des classes dérivées (*Derived Classes*). En C#, l'héritage d'implémentation est **simple** (une classe ne peut hériter que d'une seule classe mère) et s'indique au niveau de la déclaration avec le symbole deux-points (:).

Cette architecture réduit la duplication de code et facilite la maintenance des systèmes à large échelle (comme un système de routage HTTP où tous les routeurs héritent d'une base commune).

13.2 Méthodes Abstraites, Virtuelles et Scellement

Le comportement de l'héritage est rigoureusement contrôlé par des modificateurs spécifiques :

- **abstract** : Une classe abstraite est incomplète et ne peut pas être instanciée (l'appel à **new** est interdit). Elle sert de fondation contractuelle. Une méthode **abstract** n'a pas de corps et **doit impérativement** être implémentée par l'enfant.
- **virtual** : Une méthode virtuelle possède une implémentation par défaut exploitable dans la classe mère, mais **peut** être redéfinie (surchargée) de manière optionnelle par l'enfant via le mot-clé **override**.
- **base** : Permet à une classe enfant d'appeler explicitement l'implémentation de sa classe mère. Très utilisé dans les constructeurs ou pour enrichir une méthode **virtual** existante plutôt que de l'écraser complètement.
- **sealed** : Appliqué à une classe, il bloque définitivement l'héritage (aucune autre classe ne peut en dériver). Appliqué à une méthode **override**, il empêche les enfants sous-jacents de la redéfinir à nouveau. Utilisé pour des raisons de sécurité ou de micro-optimisation de compilation.

13.3 Le Polymorphisme et le Transtypage (*Casting*)

Le **polymorphisme** est la capacité d'une instance dérivée à être manipulée via une variable typée comme son parent, tout en garantissant que c'est bien le code spécifique à son type réel qui sera exécuté à l'exécution (résolution dynamique).

Pour vérifier dynamiquement la nature d'un objet en mémoire, le C# offre des opérateurs de transtypage :

- **is** : Vérifie si un objet est compatible avec un type donné (Pattern Matching).
- **as** : Tente une conversion sécurisée vers un type de référence. Retourne **null** si la conversion échoue, évitant ainsi un crash applicatif.

13.4 Exemple de Code Commenté

L'exemple suivant modélise un pipeline de traitement de documents (PDF, XML) démontrant l'héritage et l'exécution polymorphe.

```
1 // 1. Classe de base abstraite
2 public abstract class DocumentProcessor
3 {
4     protected string ParserVersion = "1.0";
5
6     // 2. Méthode virtuelle : comportement par défaut surchargeable
7     public virtual void Validate(byte[] data)
8     {
9         if (data == null || data.Length == 0)
10             throw new ArgumentNullException(nameof(data), "Le flux
11 binaire est vide.");
12     }
13
14     // 3. Méthode abstraite : contrat strict sans corps
15     public abstract void Process(byte[] data);
16 }
17
18 // 4. Héritage via le symbole ':'
19 public class PdfProcessor : DocumentProcessor
20 {
21     // 5. Redéfinition obligatoire de la méthode abstraite
```

```

21 // 6. 'sealed' bloque la redéfinition pour de futurs enfants
22 public sealed override void Process(byte[] data)
23 {
24     // 7. Appel à l'implémentation parent de validation avec 'base'
25     base.Validate(data);
26     Console.WriteLine($"Analyse du flux PDF (Moteur v{ParserVersion
27     });");
28 }
29
30 public class XmlProcessor : DocumentProcessor
31 {
32     // Redéfinition optionnelle de la méthode virtuelle
33     public override void Validate(byte[] data)
34     {
35         base.Validate(data);
36         Console.WriteLine("Vérification approfondie du schéma XSD.");
37     }
38
39     public override void Process(byte[] data)
40     {
41         Console.WriteLine("Désérialisation des noeuds XML.");
42     }
43 }
44
45 class Program
46 {
47     static void Main()
48     {
49         // 8. Polymorphisme : Un tableau parent stockant des enfants hétérogènes
50         DocumentProcessor[] processors = {
51             new PdfProcessor(),
52             new XmlProcessor()
53         };
54
55         byte[] fakePayload = new byte[256];
56
57         foreach (var proc in processors)
58         {
59             // Résolution dynamique de la bonne méthode Process() au
runtime
60             proc.Process(fakePayload);
61
62             // 9. Transtypage avec 'is' (Pattern Matching moderne)
63             if (proc is XmlProcessor xmlProc)
64             {
65                 Console.WriteLine("-> Type XML détecté, routage vers l'
API externe.");
66             }
67
68             // Alternative historique avec 'as'
69             PdfProcessor pdfProc = proc as PdfProcessor;
70             if (pdfProc != null)
71             {
72                 Console.WriteLine("-> Type PDF détecté, génération des
vignettes.");
73             }

```

```

74     }
75 }
76 }

```

Listing 34 – Système de traitement documentaire avec polymorphisme

Explications ligne par ligne :

- **Ligne 14** : Définition d'un comportement abstrait. Tout `DocumentProcessor` doit savoir se traiter (`Process`), mais l'algorithme exact dépend intimement du format concret.
- **Ligne 22** : `PdfProcessor` satisfait le contrat en redéfinissant `Process` avec `override`. Le mot-clé `sealed` garantit la fin de la chaîne d'héritage pour ce comportement.
- **Ligne 25** : `base.Validate()` déclenche la vérification de nullité implémentée en ligne 7, évitant de dupliquer ce code de sécurité métier.
- **Ligne 51** : Le tableau est typé `DocumentProcessor`, mais il contient en mémoire des instances réelles `PdfProcessor` et `XmlProcessor`. C'est la force du couplage lâche.
- **Ligne 60** : L'opérateur `is` effectue un test de type et affecte directement la variable typée `xmlProc` si le test réussit, évitant un `cast` manuel redondant.

Pièges courants & Erreurs de compilation

- **`InvalidCastException`** : Tenter de forcer un transtypage de manière brutale avec des parenthèses (ex : `(PdfProcessor)proc`) lèvera une exception violente si `proc` est en réalité un `XmlProcessor`. Il faut **toujours** privilégier les opérateurs `is` ou `as`.
- **Masquage accidentel (*Hiding*)** : Si l'enfant déclare une méthode avec la même signature que le parent sans utiliser `override` (mais avec `new`), la liaison polymorphique est brisée. Le compilateur déclenchera l'avertissement CS0114.

13.5 Synthèse / À retenir

- **Factorisation** : Poussez le code commun vers le haut (dans la classe de base) pour respecter le principe DRY (*Don't Repeat Yourself*).
- **Abstract vs Virtual** : Utilisez `abstract` pour forcer le développeur enfant à écrire du code. Utilisez `virtual` pour proposer une implémentation par défaut optionnellement modifiable.
- **Couplage Faible** : Typisez vos variables, vos paramètres de méthodes et vos collections avec le type de la classe de base, pas avec le type des enfants, afin de concevoir une architecture ouverte à l'ajout futur de nouveaux comportements.

14 POO — Interfaces

14.1 Contrat de Service : L'Interface

En ingénierie logicielle C#, une **interface** définit un contrat strict composé exclusivement de signatures (méthodes, propriétés, événements) sans fournir, traditionnellement, de logique d'implémentation ou de gestion d'état mémoire. (Les interfaces C# 8+ tolèrent des implémentations par défaut, mais ce mécanisme reste réservé à la rétrocompatibilité d'API).

Toute classe (ou structure) qui déclare implémenter une interface s'engage formellement et contractuellement à fournir l'implémentation de **tous** les membres définis par ce contrat. Contrairement à l'héritage de classes qui est limité à un seul parent, le C# autorise l'**implémentation multiple** d'interfaces, permettant à un même composant de revêtir de multiples capacités.

14.2 Interfaces Standards du Framework : IComparable et IEquatable

Le framework .NET repose massivement sur les interfaces pour connecter vos objets personnalisés aux algorithmes internes du système (tri, recherche en dictionnaire, etc.) :

- **IComparable<T>** : Utilisée pour définir une logique de **tri naturel**. Dès qu'un type l'implémente, des méthodes génériques comme `List.Sort()` deviennent fonctionnelles. Impose l'implémentation de `CompareTo(T other)`.
- **IEquatable<T>** : Utilisée pour définir l'égalité de **valeur métier** (par opposition à l'égalité d'adresse mémoire pointer-based). Elle est fortement recommandée pour les types **struct** pour éviter des pénalités de performance liées au boxing. Impose l'implémentation de `Equals(T other)`.

14.3 Interface vs Classe Abstraite : Le choix architectural

La frontière peut sembler floue, mais leur intention d'ingénierie est radicalement différente :

- **Classe Abstraite** : Modélise la nature intrinsèque d'un objet (Une *TransactionHttp* est une *Transaction*). Elle permet de partager du code métier commun et de maintenir un état interne (champs de classe). Limité à l'héritage simple.
- **Interface** : Modélise une capacité, une compétence ou un comportement transversal (Une *TransactionHttp* est **capable** d'être observée, donc implémente **Iobservable**). Elle est totalement dépourvue d'état interne. Permet l'implémentation multiple.

14.4 Exemple de Code Commenté

L'exemple suivant définit un composant de cache en mémoire illustrant l'implémentation multiple, ainsi qu'une entité métier implémentant **IComparable<T>** pour activer le tri algorithmique automatisé.

```
1 // 1. Déclaration d'interfaces (Convention .NET : préfixe 'I' majuscule)
2 public interface ICacheStorage
3 {
4     // Signature stricte, pas d'implémentation
5     void Save(string key, object data);
6     object Load(string key);
7 }
8
9 public interface IMonitorable
10 {
11     // 2. Une interface peut contracter la présence de propriétés
12     long MemoryUsageBytes { get; }
13     void PingHealthCheck();
14 }
15
16 // 3. Implémentation multiple (séparée par des virgules)
17 public class RedisCacheService : ICacheStorage, IMonitorable
18 {
19     // L'implémentation de IMonitorable.MemoryUsageBytes
20     public long MemoryUsageBytes { get; private set; }
21
22     // Implémentation obligatoire des méthodes ICacheStorage
23     public void Save(string key, object data)
24     {
25         Console.WriteLine($"[Redis] Sauvegarde du payload sur la clé : {
key}");
26         MemoryUsageBytes += 1024; // Simulation d'allocation
27     }
```

```

28
29     public object Load(string key) { return null; }
30
31     // Implémentation obligatoire de IMonitorable.PingHealthCheck
32     public void PingHealthCheck()
33     {
34         Console.WriteLine("Redis Cluster: Healthy. Latency: 4ms");
35     }
36 }
37
38 // 4. Couplage avec le moteur de tri .NET via IComparable<T>
39 public struct FinancialTransaction : IComparable<FinancialTransaction>
40 {
41     public string TransactionId { get; init; }
42     public decimal Amount { get; init; }
43
44     // 5. Contrat de tri : doit retourner 0 (égal), < 0 (avant) ou > 0 (
    après)
45     public int CompareTo(FinancialTransaction other)
46     {
47         // Pas de vérification de nullité : un struct (type valeur) ne
    peut jamais être null
48         // Délégation logique du tri directement au type décimal sous-
    jacent
49         return this.Amount.CompareTo(other.Amount);
50     }
51 }
52
53 class Program
54 {
55     static void Main()
56     {
57         // 6. Polymorphisme d'interface : Couplage faible
58         ICacheStorage cache = new RedisCacheService();
59         cache.Save("session_usr42", "{ auth: true }");
60
61         // ERREUR DE COMPILATION : 'PingHealthCheck' n'appartient pas au
    contrat ICacheStorage
62         // cache.PingHealthCheck();
63
64         // 7. Test de tri algorithmique sur type personnalisé
65         List<FinancialTransaction> transactions = new List<
    FinancialTransaction>
66         {
67             new FinancialTransaction { TransactionId = "TXN_01", Amount
    = 500.00m },
68             new FinancialTransaction { TransactionId = "TXN_02", Amount
    = 15.50m },
69             new FinancialTransaction { TransactionId = "TXN_03", Amount
    = 1200.00m }
70         };
71
72         // 8. Sort() inspecte la classe, trouve IComparable<T> et dé
    clenche CompareTo()
73         transactions.Sort();
74
75         Console.WriteLine("Historique trié par montant croissant :");
76         foreach(var t in transactions)

```

```

77         Console.WriteLine($"TXN {t.TransactionId} : {t.Amount}$");
78         // Affiche : TXN_02 (15.50), TXN_01 (500), TXN_03 (1200)
79     }
80 }

```

Listing 35 – Implémentation multiple et interfaces de tri natives

Explications ligne par ligne :

- **Ligne 2** : Convention universelle C# : toute interface doit commencer par la lettre I (pour *Interface*).
- **Ligne 17** : `RedisCacheService` s'engage contractuellement à respecter l'ensemble complet des signatures exigées par `ICacheStorage` ET par `IMonitorable`. S'il en manque une, la compilation échoue immédiatement.
- **Ligne 39** : `FinancialTransaction` implémente l'interface générique `IComparable<T>`. La généricité évite les opérations coûteuses de boxing/unboxing par rapport à la vieille interface non générique `IComparable`.
- **Ligne 60** : Le principe de couplage faible ou principe d'inversion des dépendances (le 'D' de SOLID) : le point d'entrée interagit avec l'abstraction (`ICacheStorage`) sans connaître ni se soucier de l'implémentation concrète (`RedisCacheService`).

Pièges courants & Erreurs de compilation

- **Membre non implémenté (CS0535)** : Si vous ajoutez une nouvelle signature dans l'interface `ICacheStorage`, la classe `RedisCacheService` cessera instantanément de compiler tant qu'elle ne fournira pas le code de cette nouvelle méthode.
- **Erreur de visibilité des membres** : Les méthodes implémentant une interface doivent impérativement être déclarées en `public` dans la classe (ou faire l'objet d'une implémentation d'interface explicite, une notion architecturale avancée).

14.5 Synthèse / À retenir

- **Séparation des préoccupations** : Programmez orienté *Interface* et non orienté *Implémentation*. Les paramètres de vos méthodes de service doivent attendre des interfaces (ex : `ISmtpClient`) plutôt que des classes figées.
- Les classes modélisent ce qu'une donnée **EST**, les interfaces modélisent ce dont un objet **EST CAPABLE**.
- Capitalisez massivement sur `IEquatable<T>` pour redéfinir la notion d'égalité métier et sur `IComparable<T>` pour les algorithmes de tri .NET natifs.

15 Les Génériques

15.1 Introduction Théorique

Les génériques (*Generics*) constituent un mécanisme fondamental du système de types C# permettant de définir des classes, des structures, des interfaces et des méthodes dont un ou plusieurs types sont **paramétrés**. Au lieu de figer un type concret dans le code, le développeur déclare un espace réservé (conventionnellement noté **T**) qui sera substitué par un type réel lors de l'utilisation.

Ce mécanisme élimine deux problèmes majeurs de l'ingénierie logicielle :

- **La duplication de code** : Sans génériques, il faudrait écrire une version de chaque algorithme pour chaque type de données (une méthode de tri pour **int**, une autre pour **string**, une autre pour **decimal**).
- **Le boxing/unboxing** : Avant les génériques (C# 1.0), les collections comme **ArrayList** stockaient des **object**, forçant des conversions coûteuses en performance CPU entre types valeur et types référence.

Les collections que vous avez déjà utilisées (**List<T>**, **Dictionary<TKey, TValue>**) sont elles-mêmes des classes génériques fournies par le framework .NET.

15.2 Syntaxe et Règles

15.2.1 Méthodes Génériques

Une méthode générique déclare un paramètre de type entre chevrons (**<T>**) juste après son nom. Ce paramètre devient un type utilisable dans la signature et le corps de la méthode. Le compilateur infère automatiquement le type concret à partir des arguments passés à l'appel.

15.2.2 Classes et Structures Génériques

Une classe générique déclare ses paramètres de type au niveau de la classe elle-même. Toutes les méthodes et propriétés internes peuvent alors exploiter ces paramètres de type sans les redéclarer.

15.2.3 Contraintes de Type (**where**)

Par défaut, un paramètre de type **T** peut être substitué par n'importe quel type existant, ce qui limite les opérations possibles sur **T** (on ne peut appeler que les méthodes de **object**). Les **contraintes** restreignent les types autorisés et débloquent des fonctionnalités supplémentaires :

Tableau : Contraintes de type les plus courantes

Contrainte	Effet
where T : class	T doit être un type référence (classe, interface, delegate).
where T : struct	T doit être un type valeur non nullable.
where T : new()	T doit posséder un constructeur public sans paramètre. Permet d'écrire new T() dans le code.
where T : IComparable<T>	T doit implémenter l'interface spécifiée, donnant accès à ses méthodes.
where T : BaseClass	T doit hériter de la classe de base spécifiée.
where T : notnull	T ne peut pas être null (type valeur ou type référence non nullable).

15.3 Exemple de Code Commenté

L'exemple suivant implémente un composant de cache générique en mémoire avec politique d'expiration, démontrant les méthodes génériques, les classes génériques et les contraintes de type.

```
1 // 1. Méthode générique simple : Recherche du maximum dans un tableau
2 // Contrainte : T doit implémenter IComparable<T> pour pouvoir être
   comparé
3 public static T TrouverMaximum<T>(T[] valeurs) where T : IComparable<T>
4 {
5     if (valeurs.Length == 0)
6         throw new ArgumentException(nameof(valeurs), "Le tableau ne peut
           pas être vide.");
7
8     T max = valeurs[0];
9     for (int i = 1; i < valeurs.Length; i++)
10    {
11        // CompareTo() est accessible grâce à la contrainte IComparable<
T>
12        if (valeurs[i].CompareTo(max) > 0)
13            max = valeurs[i];
14    }
15    return max;
16 }
17
18 // Appel : le compilateur infère T = int automatiquement
19 int[] mesures = { 42, 17, 93, 8, 65 };
20 int plusGrand = TrouverMaximum(mesures); // 93
21
22 // Appel explicite avec un autre type
23 string[] serveurs = { "alpha", "zulu", "bravo" };
24 string dernier = TrouverMaximum<string>(serveurs); // "zulu" (ordre
   lexicographique)
```

Listing 36 – Méthode générique et classe générique avec contraintes

```
1 // 2. Classe générique avec deux paramètres de type
2 // Contrainte : TKey doit être non null (clé de cache), TValue doit
   avoir un constructeur
3 public class MemoryCache<TKey, TValue>
4     where TKey : notnull
5     where TValue : new()
6 {
7     // Le dictionnaire interne utilise les paramètres de type de la
   classe
8     private readonly Dictionary<TKey, (TValue Data, DateTime Expiration)
> _store = new();
9
10    // 3. Méthode utilisant les paramètres de type de la classe
11    public void Set(TKey key, TValue value, TimeSpan ttl)
12    {
13        _store[key] = (value, DateTime.UtcNow.Add(ttl));
14    }
15
16    // 4. Retourne la valeur ou une nouvelle instance par défaut (grâce
   à new())
17    public TValue GetOrDefault(TKey key)
18    {
```



```

19         if (_store.TryGetValue(key, out var entry) && entry.Expiration >
20             DateTime.UtcNow)
21         {
22             return entry.Data;
23         }
24         return new TValue(); // Possible uniquement grâce à la
25         contrainte 'where TValue : new()'
26     }
27
28     public int Count => _store.Count;
29 }
30
31 class Program
32 {
33     static void Main()
34     {
35         // 5. Instanciation : substitution de TKey par string, TValue
36         par List<string>
37         var cache = new MemoryCache<string, List<string>>();
38
39         var permissions = new List<string> { "READ", "WRITE", "ADMIN" };
40         cache.Set("user_42_perms", permissions, TimeSpan.FromMinutes(30)
41     );
42
43         List<string> perms = cache.GetOrDefault("user_42_perms");
44         Console.WriteLine($"Permissions en cache : {string.Join(", ",
45             perms)}");
46
47         // 6. Tentative avec une clé expirée ou inexistante
48         List<string> inconnu = cache.GetOrDefault("user_99_perms");
49         Console.WriteLine($"Résultat par défaut : {inconnu.Count} élé
50             ments"); // 0 (new List<string>())
51     }
52 }

```

Listing 37 – Classe générique : Cache en mémoire avec expiration

Explications ligne par ligne :

- **Ligne 3** : La méthode `TrouverMaximum<T>` déclare un paramètre de type `T` contraint par `IComparable<T>`. Sans cette contrainte, l'appel à `CompareTo()` provoquerait une erreur de compilation car le compilateur ne saurait pas que `T` possède cette méthode.
- **Ligne 19** : Le compilateur `C#` infère automatiquement `T = int` à partir du type du tableau passé en argument. L'écriture explicite `TrouverMaximum<int>(mesures)` est équivalente mais redondante.
- **Lignes 28-30** : La classe `MemoryCache` déclare deux paramètres de type avec des contraintes distinctes. `notnull` garantit que la clé de cache ne sera jamais nulle. `new()` autorise l'instanciation d'un `TValue` par défaut.
- **Ligne 35** : Le dictionnaire interne utilise un tuple nommé (`TValue Data`, `DateTime Expiration`) pour stocker la valeur et sa date d'expiration. Les paramètres de type `TKey` et `TValue` de la classe sont directement exploitables dans ses membres.
- **Ligne 43** : L'instruction `new TValue()` est uniquement possible grâce à la contrainte `where TValue : new()`. Sans cette contrainte, le compilateur refuserait l'instanciation car il ne peut pas garantir que `TValue` possède un constructeur sans paramètre.

Pièges courants & Erreurs de compilation

- CS0314 — **Contrainte manquante** : Tenter d'appeler `CompareTo()`, `Equals()` ou toute méthode spécifique à une interface sur un `T` non contraint. Le compilateur ne voit que les méthodes de `object` (`ToString()`, `GetHashCode()`).
- CS0310 — **`new()` impossible** : Écrire `new T()` sans avoir déclaré la contrainte `where T : new()` dans la signature.
- **Confusion class vs struct** : Appliquer `where T : class` empêche l'utilisation de types valeur (`int`, `decimal`, `struct` personnalisées). Inversement, `where T : struct` interdit les types référence. Choisissez la contrainte en fonction de votre besoin d'allocation mémoire.

15.4 Synthèse / À retenir

- **Réutilisabilité** : Les génériques permettent d'écrire un algorithme ou une structure de données **une seule fois** pour un nombre illimité de types.
- **Sécurité à la compilation** : Contrairement à l'ancienne approche `object` + `cast`, les erreurs de type sont détectées par le compilateur et non à l'exécution.
- **Performance** : Les génériques éliminent le boxing/unboxing pour les types valeur, évitant des allocations mémoire superflues sur le tas.
- **Contraintes** : Utilisez `where` pour restreindre les types autorisés et débloquer l'accès aux méthodes d'interfaces ou de classes de base.
- **Inférence** : Le compilateur déduit automatiquement le type `T` à partir des arguments. L'écriture explicite n'est nécessaire que lorsqu'il y a ambiguïté.